

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

VÕ THỊ NGỌC HÂN

ĐỒ ÁN MÔN HỌC

THIẾT KẾ VÀ HIỆN THỰC VI XỬ LÝ RISC-V ĐA
NHÂN VỚI CƠ CHẾ ĐỒNG NHẤT BỘ NHỚ ĐỆM

DESIGN AND HARDWARE IMPLEMENTATION OF A
MULTI-CORE RISC-V PROCESSOR WITH A CACHE
COHERENCE PROTOCOL

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

VÕ THỊ NGỌC HÂN – 23520441

ĐỒ ÁN MÔN HỌC
THIẾT KẾ VÀ HIỆN THỰC VI XỬ LÝ RISC-V ĐA
NHÂN VỚI CƠ CHẾ ĐỒNG NHẤT BỘ NHỚ ĐỆM
**DESIGN AND HARDWARE IMPLEMENTATION OF A
MULTI-CORE RISC-V PROCESSOR WITH A CACHE
COHERENCE PROTOCOL**

GIẢNG VIÊN HƯỚNG DẪN
TS. TRẦN QUANG NGUYỄN

TP. HỒ CHÍ MINH, 2026

MỤC LỤC

Chương 1.	GIỚI THIỆU ĐỀ TÀI.....	3
1.1.	Tổng quan đề tài	3
1.2.	Mục tiêu đề tài	4
1.2.1.	Mục tiêu đề tài	4
1.2.2.	Chi tiết về mục tiêu đề tài.....	4
Chương 2.	CƠ SỞ LÝ THUYẾT	5
2.1.	Tổng quan về kiến trúc RISC-V và tập lệnh RV32I	5
2.1.1.	Giới thiệu về kiến trúc RISC-V.....	5
2.1.2.	Kiến trúc tập lệnh RV32I	5
2.2.	Kiến trúc vi xử lý Single-Cycle.....	9
2.2.1.	Khái niệm kiến trúc Single-Cycle	9
2.2.2.	Đường dữ liệu (Datapath).....	10
2.3.	Các khối chức năng trong CPU	11
2.3.1.	Program Counter (PC).....	11
2.3.2.	Instruction Memory	12
2.3.3.	Control Unit.....	12
2.3.4.	Register File	15
2.3.5.	Immediate Generator (ImmGen)	17
2.3.6.	Arithmetic Logic Unit (ALU)	18
2.3.7.	Branch Comparator (BRC).....	19
2.3.8.	Load Store Unit (LSU).....	20
2.3.9.	Memory	22
Chương 3.	THIẾT KẾ HỆ THỐNG.....	24

3.1.	Tổng quan kiến trúc hệ thống.....	24
3.2.	Thiết kế bộ xử lý RISC-V Single-Cycle.....	25
3.2.1.	Tầng Fetch.....	25
3.2.2.	Tầng Decode.....	26
3.2.3.	Tầng Execute	27
3.2.4.	Tầng Memory Access.....	28
3.2.5.	Tầng Write Back	29
Chương 4.	KẾT QUẢ THỰC NGHIỆM	31
4.1.	Môi trường thực nghiệm.....	31
4.2.	Kịch bản kiểm thử	33
4.2.1.	Kiểm thử tập lệnh bằng Driver và Scoreboard.....	33
4.2.2.	Kiểm thử tích hợp Memory-Mapped I/O	37
4.2.3.	Kiểm thử in chuỗi HELLO.....	40
Chương 5.	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	46
5.1.	Kết luận.....	46
5.2.	Hướng phát triển trong tương lai.....	46

DANH MỤC HÌNH ẢNH

Hình 2.1: Cấu trúc tập lệnh RV32I	6
Hình 2.2: Tập lệnh cơ bản của RV32I	7
Hình 2.3: Nhóm lệnh logic và số học.....	8
Hình 2.4: Nhóm lệnh nhảy và rẽ nhánh	8
Hình 2.5: Nhóm lệnh Load và Store	9
Hình 2.6: Đường dữ liệu của bộ xử lý RISC-V Single-Cycle	10
Hình 2.7: Sơ đồ khối Program Counter	12
Hình 2.8: Sơ đồ khối Control Unit.....	13
Hình 2.9: Kiến trúc tổng quan khối Control Unit	13
Hình 2.10: Waveform khối Control Unit.....	15
Hình 2.11: Sơ đồ khối RegFile.....	16
Hình 2.12: Waveform khối Register File.....	16
Hình 2.13: Sơ đồ khối Immgen.....	17
Hình 2.14: Sơ đồ tổng quát khối ALU.....	18
Hình 2.15: Waveform khối ALU	19
Hình 2.16: Sơ đồ khối BRC	19
Hình 2.17: Sơ đồ khối LSU.....	20
Hình 2.18: Waveform khối LSU	22
Hình 2.19: Sơ đồ khối Memory	23
Hình 3.1: Tổng quan kiến trúc hệ thống	24
Hình 3.2: Kiến trúc tầng Fetch.....	25
Hình 3.3: Kiến trúc tầng Decode.....	26
Hình 3.4: Kiến trúc tầng Execute.....	28

Hình 3.5: Kiến trúc tầng Memory Access.....	29
Hình 3.6: Kiến trúc tầng Write Back	30
Hình 4.1: Flowchart thực thi của Testbench	32
Hình 4.2: Waveform kiểm tra testbench1	34
Hình 4.3: Kết quả mô phỏng	35
Hình 4.4: Kết quả mô phỏng cơ chế Memory-Mapped I/O.....	39
Hình 4.5: Kết quả hiển thị ký tự “H”	41
Hình 4.6: Kết quả hiển thị ký tự “E”	42
Hình 4.7: Kết quả hiển thị ký tự “L”	42
Hình 4.8: Kết quả hiển thị ký tự “L”	43
Hình 4.9: Kết quả hiển thị ký tự “O”	44
Hình 4.10: Kết quả hoàn thành mô phỏng	44

DANH MỤC BẢNG

Bảng 2.1: Bảng giải mã khối Main Decoder.....	14
Bảng 2.2: Bảng giải mã opcode khối ALU Decoder	14
Bảng 2.3: Opcode các phép toán khối ALU	19
Bảng 2.4: LSU Memory Mapping	21
Bảng 2.5: Các cổng Input/Output của khối LSU và ngoại vi	21
Bảng 4.1: Thông tin hệ thống sử dụng.....	31
Bảng 4.2: Kết quả kiểm thử các lệnh của bộ xử lý RISC-V Single-Cycle	36
Bảng 4.3: Chương trình kiểm thử truy cập I/O thông qua cơ chế Memory-Mapped I/O.....	37
Bảng 4.4: Chương trình kiểm thử hiển thị chuỗi “HELLO”.....	40

MỞ ĐẦU

RISC-V, một kiến trúc tập lệnh mã nguồn mở (ISA), đã và đang trở thành một xu hướng nổi bật trong lĩnh vực thiết kế vi xử lý nhờ tính linh hoạt, khả năng mở rộng và chi phí triển khai thấp. Với triết lý thiết kế đơn giản của kiến trúc Reduced Instruction Set Computing (RISC), RISC-V cho phép các nhà phát triển tùy chỉnh và mở rộng hệ thống theo nhu cầu ứng dụng. Trong đó, tập lệnh RV32I là nền tảng cơ bản của RISC-V, cung cấp khả năng xử lý 32-bit hiệu quả và phù hợp cho việc nghiên cứu cũng như phát triển các hệ thống vi xử lý hiện đại.

Đề án này trình bày quá trình thiết kế và hiện thực một hệ thống vi xử lý RISC-V đa nhân dựa trên tập lệnh RV32I, kết hợp cơ chế đồng nhất bộ nhớ đệm (Cache Coherence) nhằm đảm bảo tính nhất quán dữ liệu giữa các lõi xử lý khi cùng truy cập vào vùng nhớ dùng chung. Hệ thống được xây dựng với nhiều lõi xử lý hoạt động song song, mỗi lõi được trang bị bộ nhớ đệm riêng và được kết nối thông qua một giao thức đồng nhất bộ nhớ đệm. Cơ chế này cho phép duy trì tính nhất quán dữ liệu, giảm thiểu các lỗi phát sinh do truy cập đồng thời và nâng cao hiệu năng xử lý của toàn hệ thống.

Đề tài tập trung nghiên cứu kiến trúc đa nhân, thiết kế bộ nhớ đệm và xây dựng cơ chế đồng nhất bộ nhớ đệm phù hợp cho hệ thống RISC-V. Quá trình hiện thực bao gồm thiết kế vi xử lý RV32I, bộ nhớ đệm dữ liệu và tập lệnh, bộ điều khiển đồng nhất bộ nhớ đệm, cùng hệ thống liên kết giữa các lõi xử lý và bộ nhớ chính. Hệ thống được mô phỏng và kiểm chứng thông qua các kịch bản truy cập bộ nhớ dùng chung nhằm đánh giá tính đúng đắn của giao thức đồng nhất, khả năng xử lý song song và hiệu năng của kiến trúc đa nhân.

Kết quả đạt được cho thấy hệ thống có khả năng duy trì tính nhất quán dữ liệu giữa các lõi xử lý, hỗ trợ thực thi đồng thời nhiều tác vụ và nâng cao hiệu quả sử dụng tài nguyên phần cứng. Đề tài góp phần làm rõ các nguyên lý thiết kế vi xử lý đa nhân dựa trên kiến trúc RISC-V, đồng thời tạo tiền đề cho các nghiên cứu mở rộng về hệ thống đa lõi hiệu năng cao, giao thức đồng nhất bộ nhớ đệm tiên tiến và các ứng dụng xử lý song song trong tương lai.

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1. Tổng quan đề tài

Trong những năm gần đây, nhu cầu xử lý song song ngày càng gia tăng trong các lĩnh vực như trí tuệ nhân tạo, xử lý tín hiệu số, hệ điều hành nhúng và trung tâm dữ liệu. Việc tiếp tục nâng cao hiệu năng bằng cách tăng tần số hoạt động của bộ xử lý gặp nhiều giới hạn về công suất tiêu thụ và nhiệt lượng. Do đó, kiến trúc đa nhân (Multi-core Processor) đã trở thành xu hướng chủ đạo nhằm nâng cao hiệu năng tính toán thông qua khả năng thực thi đồng thời nhiều luồng xử lý.

Bên cạnh đó, kiến trúc tập lệnh mã nguồn mở RISC-V đang nhận được sự quan tâm lớn từ cộng đồng nghiên cứu và công nghiệp bán dẫn nhờ tính linh hoạt, khả năng mở rộng và không phụ thuộc bản quyền. Việc nghiên cứu và phát triển các hệ thống đa nhân dựa trên RISC-V góp phần tạo nền tảng cho các hệ thống xử lý hiệu năng cao trong tương lai.

Tuy nhiên, khi nhiều lõi xử lý cùng sử dụng bộ nhớ dùng chung, mỗi lõi thường được trang bị bộ nhớ đệm riêng nhằm giảm độ trễ truy cập dữ liệu. Điều này dẫn đến bài toán đồng nhất bộ nhớ đệm (Cache Coherence), trong đó các bản sao dữ liệu tồn tại trên nhiều cache có thể trở nên không nhất quán khi xảy ra thao tác ghi dữ liệu từ một lõi xử lý.

Nếu không có cơ chế đồng nhất bộ nhớ đệm, các lõi xử lý có thể đọc những giá trị khác nhau của cùng một địa chỉ bộ nhớ, gây ra lỗi trong quá trình thực thi chương trình song song. Vì vậy, việc nghiên cứu và hiện thực các giao thức đồng nhất bộ nhớ đệm đóng vai trò quan trọng trong thiết kế vi xử lý đa nhân hiện đại.

Đề tài tập trung thiết kế và hiện thực một hệ thống vi xử lý RISC-V đa nhân dựa trên tập lệnh RV32I, trong đó mỗi lõi xử lý được trang bị bộ nhớ đệm riêng và cùng truy cập đến bộ nhớ chính dùng chung. Hệ thống áp dụng cơ chế đồng nhất bộ nhớ đệm nhằm đảm bảo tính nhất quán dữ liệu giữa các lõi xử lý, đồng thời đánh

giá khả năng hoạt động của giao thức thông qua các kịch bản truy cập bộ nhớ song song.

1.2. Mục tiêu đề tài

1.2.1. Mục tiêu đề tài

Mục tiêu của đề tài là nghiên cứu, thiết kế và hiện thực một hệ thống vi xử lý RISC-V đa nhân dựa trên tập lệnh RV32I, tích hợp cơ chế đồng nhất bộ nhớ đệm nhằm đảm bảo tính nhất quán dữ liệu giữa các lõi xử lý khi truy cập bộ nhớ dùng chung. Hệ thống được xây dựng và mô phỏng ở mức RTL bằng ngôn ngữ mô tả phần cứng Verilog, đồng thời được kiểm chứng thông qua các kịch bản truy cập bộ nhớ song song nhằm đánh giá tính đúng đắn của cơ chế đồng nhất bộ nhớ đệm.

Bên cạnh đó, đề tài hướng đến việc tìm hiểu các nguyên lý thiết kế hệ thống đa nhân, kiến trúc bộ nhớ phân cấp, cơ chế hoạt động của bộ nhớ đệm và các giao thức đồng nhất bộ nhớ đệm được sử dụng phổ biến trong các bộ xử lý hiện đại. Kết quả của đề tài góp phần tạo nền tảng cho các nghiên cứu chuyên sâu về kiến trúc máy tính, hệ thống đa lõi và thiết kế vi xử lý hiệu năng cao dựa trên kiến trúc RISC-V.

1.2.2. Chi tiết về mục tiêu đề tài

- Nghiên cứu kiến trúc tập lệnh RISC-V RV32I và nguyên lý hoạt động của vi xử lý RISC-V.
- Thiết kế và hiện thực vi xử lý RISC-V đơn nhân theo kiến trúc Single-Cycle.
- Mở rộng hệ thống từ kiến trúc đơn nhân sang kiến trúc đa nhân gồm nhiều lõi RISC-V.
- Thiết kế bộ nhớ đệm dữ liệu riêng cho từng lõi xử lý.
- Thiết kế bộ nhớ dùng chung và cơ chế giao tiếp giữa các lõi xử lý.
- Thiết kế khối điều khiển đồng nhất bộ nhớ đệm (Coherence Controller).
- Hiện thực giao thức đồng nhất bộ nhớ đệm MSI.
- Xây dựng các kịch bản kiểm thử truy cập bộ nhớ đồng thời giữa các lõi xử lý.
- Đánh giá tính đúng đắn của cơ chế đồng nhất bộ nhớ đệm và hiệu quả của hệ thống đa nhân.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về kiến trúc RISC-V và tập lệnh RV32I

2.1.1. Giới thiệu về kiến trúc RISC-V

RISC-V là một kiến trúc tập lệnh mã nguồn mở (Instruction Set Architecture – ISA), nó cung cấp một tiêu chuẩn thiết kế vi xử lý miễn phí, linh hoạt và có khả năng tùy biến cao cho các hệ thống từ vi điều khiển cho đến siêu máy tính.

Kiến trúc RISC-V được xây dựng dựa trên triết lý thiết kế của kiến trúc RISC với mục tiêu đơn giản hóa phần cứng, giảm độ phức tạp trong thiết kế và nâng cao hiệu suất thực thi. RISC-V định nghĩa một tập lệnh cơ sở tối thiểu và cho phép mở rộng thông qua các tập lệnh bổ sung nhằm đáp ứng các nhu cầu ứng dụng khác nhau, từ các hệ thống nhúng công suất thấp đến các hệ thống xử lý hiệu năng cao.

RISC-V nổi bật như một kiến trúc tập lệnh mã nguồn mở (ISA), cung cấp khả năng tùy chỉnh cao hơn ARM hoặc x86. Không giống như ARM (yêu cầu cấp phép) và x86 (được kiểm soát chặt chẽ bởi Intel và AMD), RISC-V cho phép các nhà phát triển sửa đổi và mở rộng ISA một cách tự do, lý tưởng cho các thiết kế phù hợp trên các ứng dụng từ IoT đến trung tâm dữ liệu.

2.1.2. Kiến trúc tập lệnh RV32I

RV32I là tập lệnh cơ sở số nguyên 32-bit của kiến trúc RISC-V có độ rộng thanh ghi và không gian địa chỉ 32-bit. Tập lệnh được thiết kế để giảm thiểu phần cứng nhưng vẫn cung cấp đầy đủ các thành phần chức năng để tạo ra một bộ xử lý hoàn chỉnh.

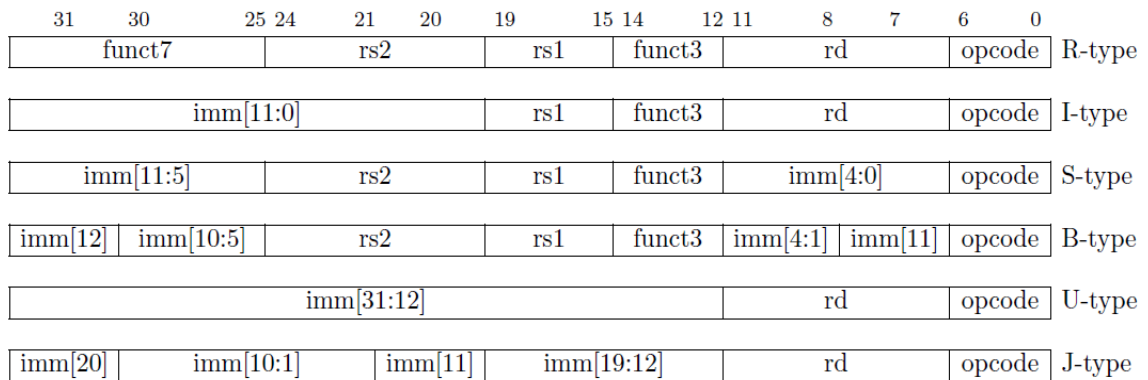
RV32I cung cấp 32 thanh ghi số nguyên, được kí hiệu từ x0 đến x31, mỗi thanh ghi có độ rộng 32-bit. Ngoài ra, còn có một thanh ghi đếm PC (Program Counter). Trong đó:

- Thanh ghi x0: giá trị luôn cố định bằng 0. Mọi thao tác ghi đều không thực hiện được với thanh ghi này.

- Các thanh ghi x1 – x31: dùng để lưu trữ toán hạng, kết quả tính toán, địa chỉ và các giá trị tạm thời trong quá trình thực thi chương trình.

Để đơn giản hoá việc giải mã phần cứng, RISC-V được chia thành 6 định dạng lệnh chính:

- R – type (Register): Dùng cho các phép toán giữa các thanh ghi.
- I – type (Immediate): Dùng cho các phép toán có hằng số hoặc lệnh Load.
- S – type (Store): Dùng cho các lệnh ghi dữ liệu vào bộ nhớ.
- B – type (Branch): Dùng cho các lệnh rẽ nhánh có điều kiện.
- U – type (Upper Immediate): Dùng cho các lệnh xử lý hằng số lớn hơn 20-bit cao.
- J – type (Jump): Dùng cho các lệnh nhảy không điều kiện.



Hình 2.1: Cấu trúc tập lệnh RV32I

RV32I Base Instruction Set								
imm[31:12]				rd		0110111	LUI rd,imm	
imm[31:12]				rd		0010111	AUIPC rd,imm	
imm[20 10:1 11 19:12]				rd		1101111	JAL rd,imm	
imm[11:0]			rs1	000	rd		1100111	JALR rd,rs1,imm
imm[12 10:5]		rs2	rs1	000	imm[4:1 11]	1100011		BEQ rs1,rs2,imm
imm[12 10:5]		rs2	rs1	001	imm[4:1 11]	1100011		BNE rs1,rs2,imm
imm[12 10:5]		rs2	rs1	100	imm[4:1 11]	1100011		BLT rs1,rs2,imm
imm[12 10:5]		rs2	rs1	101	imm[4:1 11]	1100011		BGE rs1,rs2,imm
imm[12 10:5]		rs2	rs1	110	imm[4:1 11]	1100011		BLTU rs1,rs2,imm
imm[12 10:5]		rs2	rs1	111	imm[4:1 11]	1100011		BGEU rs1,rs2,imm
imm[11:0]			rs1	000	rd		0000011	LB rd,rs1,imm
imm[11:0]			rs1	001	rd		0000011	LH rd,rs1,imm
imm[11:0]			rs1	010	rd		0000011	LW rd,rs1,imm
imm[11:0]			rs1	100	rd		0000011	LBU rd,rs1,imm
imm[11:0]			rs1	101	rd		0000011	LHU rd,rs1,imm
imm[11:5]		rs2	rs1	000	imm[4:0]	0100011		SB rs1,rs2,imm
imm[11:5]		rs2	rs1	001	imm[4:0]	0100011		SH rs1,rs2,imm
imm[11:5]		rs2	rs1	010	imm[4:0]	0100011		SW rs1,rs2,imm
imm[11:0]			rs1	000	rd		0010011	ADDI rd,rs1,imm
imm[11:0]			rs1	010	rd		0010011	SLTI rd,rs1,imm
imm[11:0]			rs1	011	rd		0010011	SLTIU rd,rs1,imm
imm[11:0]			rs1	100	rd		0010011	XORI rd,rs1,imm
imm[11:0]			rs1	110	rd		0010011	ORI rd,rs1,imm
imm[11:0]			rs1	111	rd		0010011	ANDI rd,rs1,imm
0000000		shamt	rs1	001	rd		0010011	SLLI rd,rs1,shamt
0000000		shamt	rs1	101	rd		0010011	SRLI rd,rs1,shamt
0100000		shamt	rs1	101	rd		0010011	SRAI rd,rs1,shamt
0000000		rs2	rs1	000	rd		0110011	ADD rd,rs1,rs2
0100000		rs2	rs1	000	rd		0110011	SUB rd,rs1,rs2
0000000		rs2	rs1	001	rd		0110011	SLL rd,rs1,rs2
0000000		rs2	rs1	010	rd		0110011	SLT rd,rs1,rs2
0000000		rs2	rs1	011	rd		0110011	SLTU rd,rs1,rs2
0000000		rs2	rs1	100	rd		0110011	XOR rd,rs1,rs2
0000000		rs2	rs1	101	rd		0110011	SRL rd,rs1,rs2
0100000		rs2	rs1	101	rd		0110011	SRA rd,rs1,rs2
0000000		rs2	rs1	110	rd		0110011	OR rd,rs1,rs2
0000000		rs2	rs1	111	rd		0110011	AND rd,rs1,rs2
0000	pred	succ	00000	000	00000	0001111		FENCE
0000	0000	0000	00000	001	00000	0001111		FENCE.I
000000000000			00000	000	00000	1110011		SCALL
000000000001			00000	000	00000	1110011		SBREAK
110000000000			00000	010	rd	1110011		RDCYCLE rd
110010000000			00000	010	rd	1110011		RDCYCLEH rd
110000000001			00000	010	rd	1110011		RDTIME rd
110010000001			00000	010	rd	1110011		RDTIMEH rd
110000000010			00000	010	rd	1110011		RDINSTRET rd
110010000010			00000	010	rd	1110011		RDINSTRETH rd

Hình 2.2: Tập lệnh cơ bản của RV32I

2.1.2.1. Nhóm lệnh số học và logic

Nhóm lệnh số học và logic là nhóm lệnh được sử dụng phổ biến nhất trong quá trình thực thi. Được sử dụng để thực hiện các phép toán logic trên dữ liệu được lưu trữ trong các thanh ghi. Kết quả của các phép toán này sẽ được ghi trở lại thanh ghi đích rd sau khi thực thi xong.

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srl	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	

Hình 2.3: Nhóm lệnh logic và số học

2.1.2.2. Nhóm lệnh nhảy và rẽ nhánh

Nhóm lệnh nhảy và rẽ nhánh được sử dụng để điều khiển luồng thực thi của chương trình. Có 2 nhóm lệnh được sử dụng: B-type và J-type. Trong đó, B-type dùng cho việc rẽ nhánh có điều kiện, nếu thoả mãn điều kiện, lệnh rẽ nhánh sẽ được thực thi, nếu không thoả mãn thì không rẽ nhánh. J-type là nhóm lệnh rẽ nhánh không điều kiện, tức là lệnh sẽ luôn được thực thi nhảy tới một vị trí cụ thể mà không cần thoả mãn bất kì điều kiện nào. Hai lệnh này đồng thời lưu địa chỉ lệnh kế tiếp vào thanh ghi đích để quay trở lại chương trình sau khi thực hiện lời gọi hàm.

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 ≥ rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 ≥ rs2) PC += imm	zero-extends
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	

Hình 2.4: Nhóm lệnh nhảy và rẽ nhánh

2.1.2.3. Nhóm lệnh Load và Store

Trong kiến trúc RISC-V, việc truy cập bộ nhớ chỉ được thực hiện thông qua lệnh Load và Store: S-type và I-type. Trong đó, S-type là nhóm lệnh dùng để lưu trữ dữ liệu từ thanh ghi vào bộ nhớ, gồm các lệnh theo từng kích thước dữ liệu như SB

(Store Byte), SH (Store Half), SW (Store Word). I-type là nhóm lệnh dùng để load dữ liệu từ bộ nhớ ra thanh ghi, gồm các lệnh như LB (Load Byte), LH (Load Half), LW (Load Word), LBU (Load Byte Unsigned), LHU (Load Half Unsigned). Địa chỉ bộ nhớ được tính bằng cách cộng giá trị trong thanh ghi cơ sở rs1 với giá trị immediate của lệnh.

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
lb	Load Byte	I	0000011	0x0		$rd = M[rs1+imm][0:7]$	
lh	Load Half	I	0000011	0x1		$rd = M[rs1+imm][0:15]$	
lw	Load Word	I	0000011	0x2		$rd = M[rs1+imm][0:31]$	
lbu	Load Byte (U)	I	0000011	0x4		$rd = M[rs1+imm][0:7]$	zero-extends
lhu	Load Half (U)	I	0000011	0x5		$rd = M[rs1+imm][0:15]$	zero-extends
sb	Store Byte	S	0100011	0x0		$M[rs1+imm][0:7] = rs2[0:7]$	
sh	Store Half	S	0100011	0x1		$M[rs1+imm][0:15] = rs2[0:15]$	
sw	Store Word	S	0100011	0x2		$M[rs1+imm][0:31] = rs2[0:31]$	

Hình 2.5: Nhóm lệnh Load và Store

2.2. Kiến trúc vi xử lý Single-Cycle

2.2.1. Khái niệm kiến trúc Single-Cycle

Kiến trúc Single-Cycle là kiến trúc thực thi một lệnh hoàn chỉnh chỉ trong một chu kỳ xung nhịp. Các giai đoạn của quá trình thực thi một lệnh bao gồm: Instruction Fetch, Instruction Decode, Execute, Memory Access và Write Back đều được hoàn thành trong một chu kỳ xung nhịp.

Trong kiến trúc Single-Cycle, tất cả các khối chức năng của bộ xử lý hoạt động đồng thời trong một chu kỳ clock. Tại cạnh lên của clock, bộ xử lý bắt đầu thực hiện một lệnh mới và hoàn thành trước cạnh lên của clock tiếp theo.

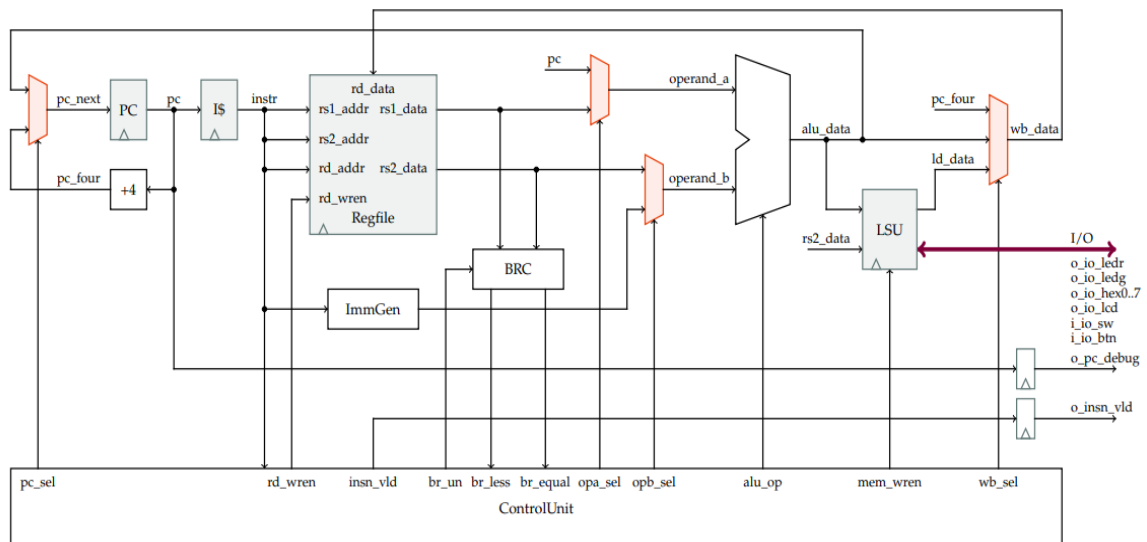
Tuy nhiên, do tất cả các lệnh đều phải hoàn thành trong một chu kỳ clock nên chu kỳ xung nhịp phải được lựa chọn đủ lớn để đáp ứng lệnh có thời gian thực thi dài nhất. Điều này dẫn đến việc nhiều lệnh đơn giản như phép cộng hoặc phép logic phải sử dụng cùng một chu kỳ clock dài như các lệnh truy cập bộ nhớ, làm giảm hiệu năng tổng thể của hệ thống.

Mặc dù tồn tại một số hạn chế về hiệu năng, kiến trúc Single-Cycle vẫn được sử dụng rộng rãi trong các hệ thống giáo dục và nghiên cứu nhờ tính trực quan và khả năng hiện thực đơn giản.

2.2.2. Đường dữ liệu (Datapath)

Datapath là tập hợp các khối phần cứng chịu trách nhiệm lưu trữ, xử lý và truyền dữ liệu bên trong bộ xử lý. Datapath xác định đường đi của dữ liệu giữa các khối chức năng trong quá trình thực thi lệnh, đồng thời phối hợp với khối điều khiển để thực hiện các phép toán tương ứng với từng loại lệnh.

Trong kiến trúc Single-Cycle, toàn bộ quá trình thực thi một lệnh được hoàn thành trong một chu kỳ xung nhịp. Do đó, đường dữ liệu phải cung cấp đầy đủ các đường truyền và khối xử lý cần thiết để hỗ trợ các nhóm lệnh số học – logic, truy cập bộ nhớ, rẽ nhánh và nhảy.



Hình 2.6: Đường dữ liệu của bộ xử lý RISC-V Single-Cycle

Datapath trong Hình 2.6 bao gồm các khối chức năng chính: Program Counter (PC), bộ cộng PC + 4, Instruction Memory (IMEM), Register File (RegFile), Immediate Generator (ImmGen), Branch Comparator (BRC), Arithmetic Logic Unit (ALU), Load Store Unit (LSU), các bộ Multiplexer và khối Control Logic.

Quá trình thực thi một lệnh trên datapath có thể được mô tả qua năm bước chức năng như sau:

- Bước 1 – Instruction Fetch: PC cung cấp địa chỉ cho Instruction Memory để lấy mã lệnh. Đồng thời, bộ cộng tính $PC + 4$. Bộ Multiplexer đầu vào PC lựa chọn giữa $PC + 4$ và địa chỉ đích do ALU tạo ra để xác định địa chỉ lệnh tiếp theo.
- Bước 2 – Instruction Decode: Mã lệnh được phân tách thành các trường chức năng. Register File đọc dữ liệu từ hai thanh ghi nguồn rs1 và rs2; Immediate Generator tạo giá trị immediate; Control Logic giải mã lệnh và sinh các tín hiệu điều khiển cho datapath.
- Bước 3 – Execute: Các Multiplexer lựa chọn toán hạng cho ALU từ PC, dữ liệu rs1, dữ liệu rs2 hoặc immediate. ALU thực hiện phép toán theo tín hiệu ALUSel. Đối với lệnh rẽ nhánh, Branch Comparator so sánh hai thanh ghi nguồn và gửi kết quả đến Control Logic để xác định giá trị PCSel.
- Bước 4 – Memory Access: Đối với lệnh Load và Store, kết quả ALU được sử dụng làm địa chỉ truy cập Load Store Unit. Lệnh Store ghi dữ liệu từ rs2 vào bộ nhớ, trong khi lệnh Load đọc dữ liệu từ bộ nhớ để chuyển đến đường Write Back.
- Bước 5 – Write Back: Bộ Multiplexer Write Back lựa chọn giữa dữ liệu đọc từ Data Memory, kết quả ALU và giá trị $PC + 4$. Dữ liệu được chọn được ghi trở lại thanh ghi đích rd khi tín hiệu wb_sel được kích hoạt.

2.3. Các khối chức năng trong CPU

Bộ xử lý RISC-V Single-Cycle được xây dựng từ nhiều khối chức năng khác nhau, mỗi khối đảm nhận nhiệm vụ riêng biệt. Các khối này liên kết với nhau thông qua datapath và các tín hiệu điều khiển để thực hiện đầy đủ các chức năng của tập lệnh RV32I.

2.3.1. Program Counter (PC)

Program Counter (PC) là thanh ghi dùng để lưu địa chỉ của lệnh hiện tại đang được thực thi. Tại mỗi chu kỳ xung nhịp, giá trị của PC sẽ được sử dụng để truy cập bộ nhớ lệnh tương ứng.

Sau khi hoàn thành một lệnh, giá trị của PC sẽ được cộng 4 do mỗi lệnh có độ dài 32-bit (4 byte). Đối với các lệnh nhảy và rẽ nhánh, giá trị của PC sẽ được cập nhật dựa trên kết quả tính toán từ khối ALU và điều kiện rẽ nhánh.



Hình 2.7: Sơ đồ khối Program Counter

2.3.2. Instruction Memory

Instruction Memory là bộ nhớ dùng để lưu trữ chương trình thực thi của bộ xử lý, khối này nhận địa chỉ từ Program Counter và xuất lệnh tương ứng tại địa chỉ đó.

Trong kiến trúc Single-Cycle, bộ nhớ lệnh thường được thiết kế theo cơ chế chỉ đọc trong quá trình thực thi chương trình. Nội dung bộ nhớ có thể được khởi tạo từ các tập tin chứa mã máy được tạo bởi trình biên dịch RISC-V.

Instruction Memory đóng vai trò cung cấp lệnh cho toàn bộ bộ xử lý.

2.3.3. Control Unit

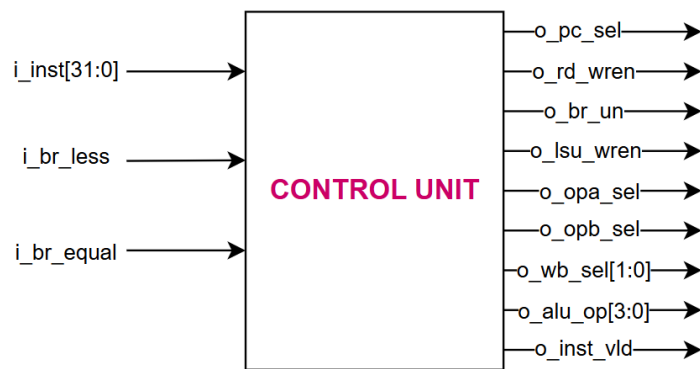
Control Unit là khối chức năng điều khiển hoạt động của toàn bộ bộ xử lý. Khối này nhận mã lệnh từ khối Instruction Memory sau đó bật các tín hiệu cần thiết để điều phối hoạt động của các khối phần cứng trong datapath.

Control Unit sử dụng opcode để xác định loại lệnh đang được thực thi và từ đó tạo ra các tín hiệu điều khiển tương ứng cho từng khối chức năng. Một số tín hiệu điều khiển quan trọng bao gồm:

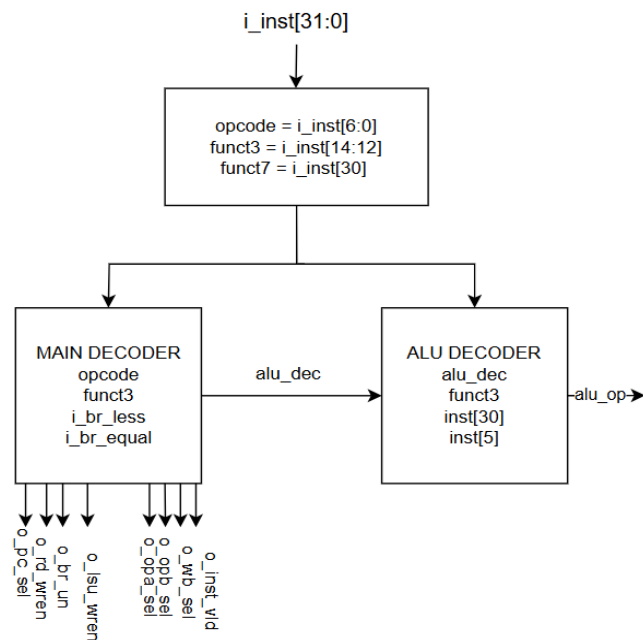
- o_pc_sel: lựa chọn giá trị cập nhật cho Program Counter.
- o_rd_wren: cho phép ghi dữ liệu vào Register File.
- o_br_un: lựa chọn kiểu so sánh có dấu hay không dấu.
- o_opa_sel: lựa chọn toán hạng thứ nhất đưa vào ALU.
- o_opb_sel: lựa chọn toán hạng thứ hai đưa vào ALU.
- o_alu_op: xác định phép toán cần thực hiện trong ALU.
- o_lsu_wren: cho phép ghi dữ liệu xuống bộ nhớ.
- o_wb_sel: lựa chọn ghi tín hiệu trở lại Register File.

Sau khi nhận lệnh từ Instruction Memory, Control Unit sẽ giải mã opcode, funct3, funct7 để xác định chức năng của lệnh. Sau đó, bật các tín hiệu tương ứng và truyền đến các khối trong datapath để thực hiện lệnh.

Do bộ xử lý được thiết kế theo kiến trúc Single-Cycle nên Control Unit được hiện thực dưới dạng logic tổ hợp (Combinational Logic), cho phép các tín hiệu điều khiển được sinh ra ngay trong cùng một chu kỳ xung nhịp mà không cần lưu trữ trạng thái trung gian.



Hình 2.8: Sơ đồ khối Control Unit



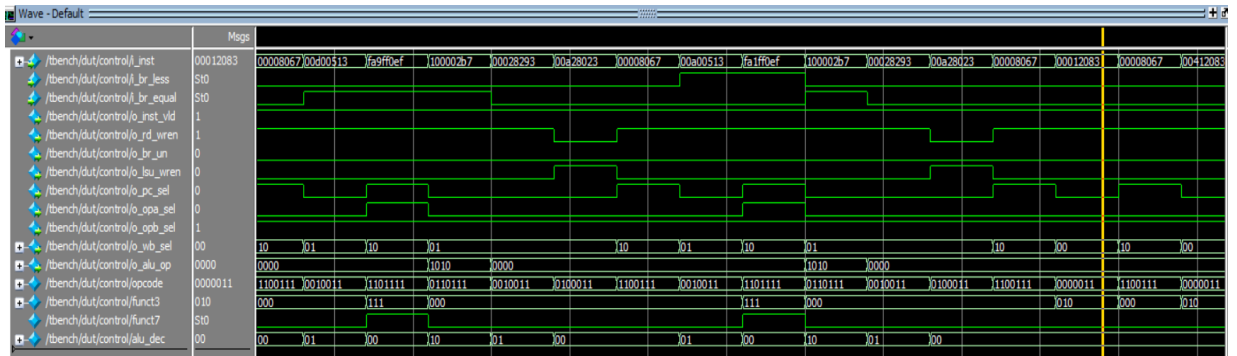
Hình 2.9: Kiến trúc tổng quan khối Control Unit

Loại lệnh	Opcode	alu_dec	Chức năng ALU
R-type	0110011	01	Giải mã phép toán theo các trường chức năng
I-type	0010011	01	Giải mã phép toán theo các trường chức năng
Load	0000011	00	Tính rs1 + immediate
Store	0100011	00	Tính rs1 + immediate
Branch	1100011	00	Tính PC + immediate
JAL	1101111	00	Tính PC + immediate
JALR	1100111	00	Tính rs1 + immediate
LUI	0110111	10	Truyền immediate
AUIPC	0010111	00	Tính PC + immediate
JALR	1100111	00	Tính rs1 + immediate

Bảng 2.1: Bảng giải mã khối Main Decoder

alu_dec	Điều kiện	o_alu_op	Phép toán
00	Không cần giải mã thêm	0000	ADD
01	funct3 = 000, SUB	0001	SUB
01	funct3 = 001	0010	SLL
01	funct3 = 010	0011	SLT
01	funct3 = 011	0100	SLTU
01	funct3 = 100	0101	XOR
01	funct3 = 101, bit30 = 0	0110	SRL
01	funct3 = 101, bit30 = 1	0111	SRA
01	funct3 = 110	1000	OR
01	funct3 = 111	1001	AND
10	LUI	1010	LUI

Bảng 2.2: Bảng giải mã opcode khối ALU Decoder



Hình 2.10: Waveform khối Control Unit

Hình 2.10 thể hiện kết quả Waveform của khối Control Unit.

Tại thời điểm lệnh 00028293, lệnh được giải mã thành `addi x5, x5, 0`. Đây là lệnh I-type, `o_rd_wren = 1` cho phép ghi kết quả về thanh ghi `x5`, `o_opa_sel = 0` chọn giá trị từ `rs1` và `o_opb_sel = 1` chọn immediate. Với `funct3 = 000`, `o_alu_op = 0000` nên ALU thực hiện phép cộng: $x5 = x5 + 0$. Vì lệnh không cần truy cập bộ nhớ nên tín hiệu `o_lsu_wren = 0`, PC tiếp tục tăng tuần tự vì `o_pc_sel = 0`.

Tại thời điểm lệnh 00a28023, đây là lệnh `sb x10, 0(x5)`. Lệnh thuộc nhóm Store với `opcode = 0100011`. Vì Store không ghi kết quả trở lại Register File nên `o_rd_wren = 0`. Ngược lại, `o_lsu_wren = 1` để cho phép LSU thực hiện ghi dữ liệu. Tín hiệu `o_opa_sel = 0` lựa chọn giá trị của `rs1 = x5`, còn `o_opb_sel = 1` lựa chọn immediate. Đồng thời `o_alu_op = 0000` để ALU thực hiện phép cộng tính địa chỉ ghi = $x5 + 0$. Sau đó, byte thấp của dữ liệu trong `x10` được ghi tới địa chỉ đã tính toán. Tại thời điểm thực hiện lệnh Store, `o_lsu_wren` chuyển lên mức 1, trong khi `o_rd_wren` chuyển xuống mức 0.

2.3.4. Register File

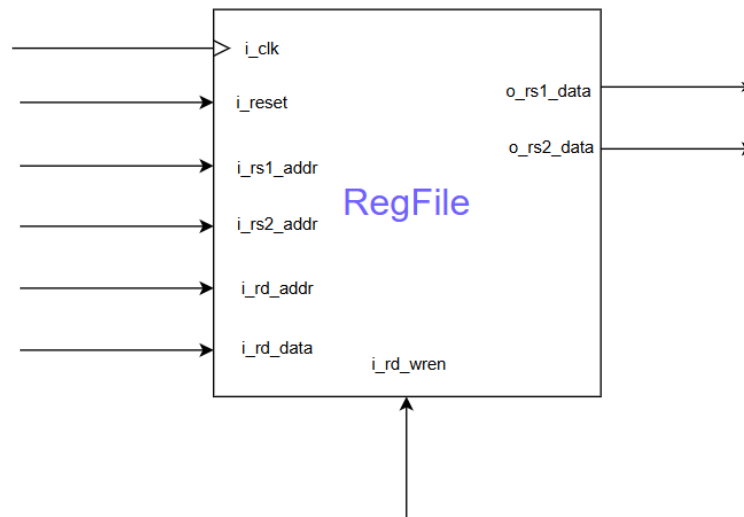
Register File là tập hợp các thanh ghi đa dụng bên trong CPU, được sử dụng để lưu trữ dữ liệu tạm thời và kết quả trung gian trong quá trình thực thi lệnh.

Kiến trúc RV32I sử dụng 32 thanh ghi đa dụng, mỗi thanh ghi có độ rộng 32-bit.

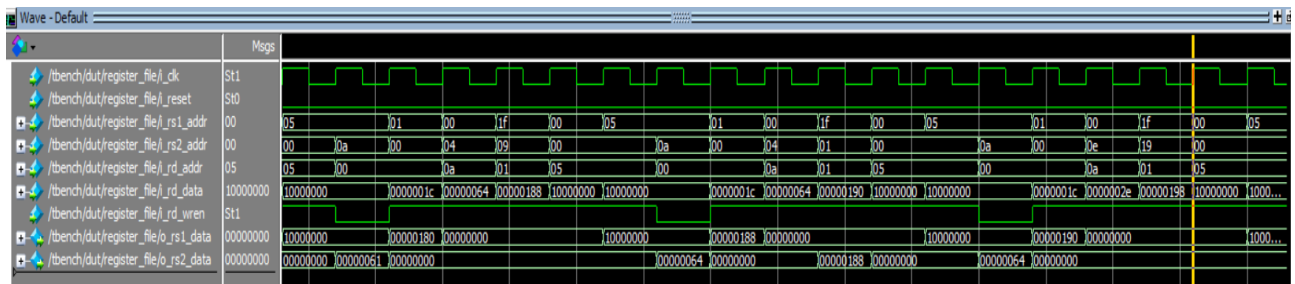
Register File thường được thiết kế với hai cổng đọc và một cổng ghi. Trong đó:

- Hai cổng đọc được sử dụng để đọc đồng thời giá trị từ hai thanh ghi nguồn rs1 và rs2.
- Một cổng ghi được sử dụng để ghi kết quả vào thanh ghi đích rd tại cạnh lên của xung clock.

Thanh ghi x0 luôn chứa giá trị bằng 0 và không thể bị ghi đè.



Hình 2.11: Sơ đồ khối RegFile



Hình 2.12: Waveform khối Register File

Hình trên thể hiện kết quả waveform khối Register File.

Waveform cho thấy khối Register File thực hiện đồng thời hai thao tác đọc và một thao tác ghi độc lập. Ban đầu, khi $i_rd_addr = 5$, $i_rd_data = 0x10000000$ và $i_rd_wren = 1$, giá trị $0x10000000$ được ghi vào thanh ghi x5 tại cạnh lên của xung clock. Đồng thời, $i_rs1_addr = 5$ nên ngõ ra o_rs1_data trả về giá trị $0x10000000$ của thanh ghi x5, trong khi $i_rs2_addr = 0$ nên $o_rs2_data = 0x00000000$ do thanh

ghi x0 luôn có giá trị bằng 0. Ở khoảng tiếp theo, i_rs2_addr chuyển sang 10, do đó o_rs2_data trả về giá trị tương ứng của thanh ghi x10, trong khi i_rd_wren = 0 nên không có thanh ghi nào được cập nhật. Ở các chu kỳ tiếp theo, khi i_rd_wren = 1, dữ liệu tại i_rd_data được ghi vào thanh ghi được xác định bởi i_rd_addr tại cạnh lên của xung clock. Trong khi đó, hai cổng đọc vẫn hoạt động độc lập và liên tục lựa chọn dữ liệu dựa trên i_rs1_addr và i_rs2_addr.

2.3.5. Immediate Generator (ImmGen)

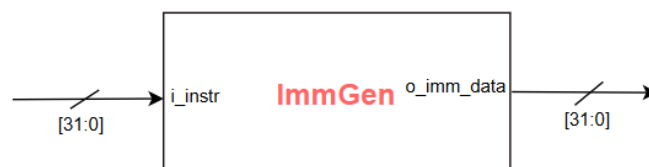
Immediate Generator (ImmGen) là khối chịu trách nhiệm trích xuất và mở rộng giá trị immediate từ lệnh hiện tại.

Do các định dạng lệnh trong RV32I có cấu trúc khác nhau nên vị trí của trường immediate cũng khác nhau. ImmGen sẽ xác định loại lệnh và thực hiện ghép các bit immediate tương ứng, đồng thời mở rộng dấu (sign extension) để tạo ra giá trị 32-bit phục vụ cho quá trình thực thi.

Khối ImmGen hỗ trợ các định dạng immediate của các loại lệnh I-Type, S-Type, B-Type, U-Type và J-Type.

Sau khi ghép các bit immediate, khối ImmGen thực hiện mở rộng dấu (Sign Extension) để tạo ra giá trị 32-bit phục vụ cho các phép tính địa chỉ, lệnh số học tức thời và các lệnh nhảy.

Giá trị immediate được truyền trực tiếp đến tầng Execute để làm một trong hai toán hạng của ALU khi cần thiết.



Hình 2.13: Sơ đồ khối Immgen

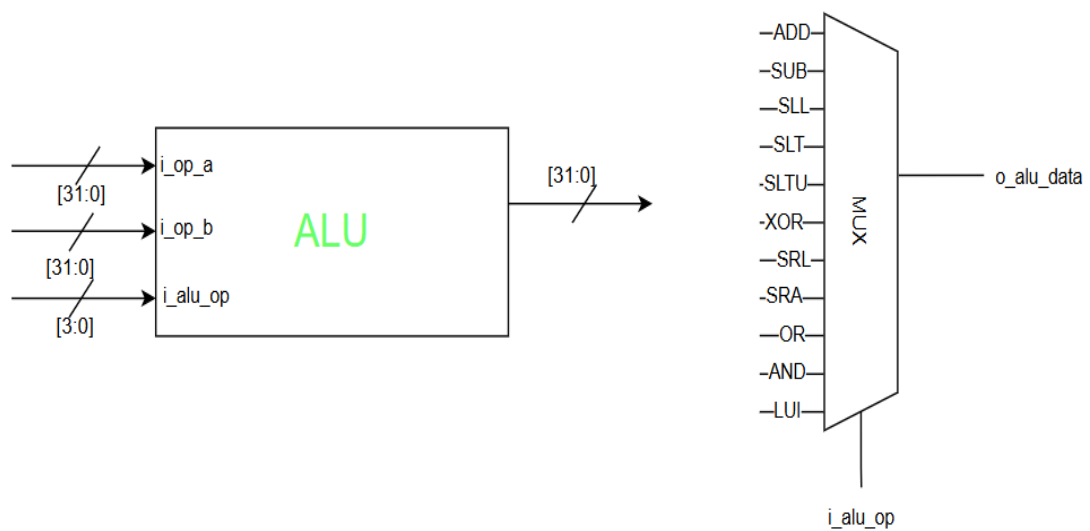
2.3.6. Arithmetic Logic Unit (ALU)

Arithmetic Logic Unit (ALU) là khối thực hiện các phép toán số học và logic trong bộ xử lý.

ALU nhận hai toán hạng đầu vào và thực hiện phép toán tương ứng dựa trên tín hiệu điều khiển từ ALU Control. Các phép toán được hỗ trợ bao gồm:

- Cộng và trừ.
- Các phép toán AND, OR, XOR.
- Dịch trái/phải.
- So sánh có dấu/không dấu.

Kết quả tính toán từ ALU được sử dụng cho nhiều mục đích khác nhau như ghi về thanh ghi đích, tính địa chỉ bộ nhớ hoặc xác định địa chỉ nhảy.

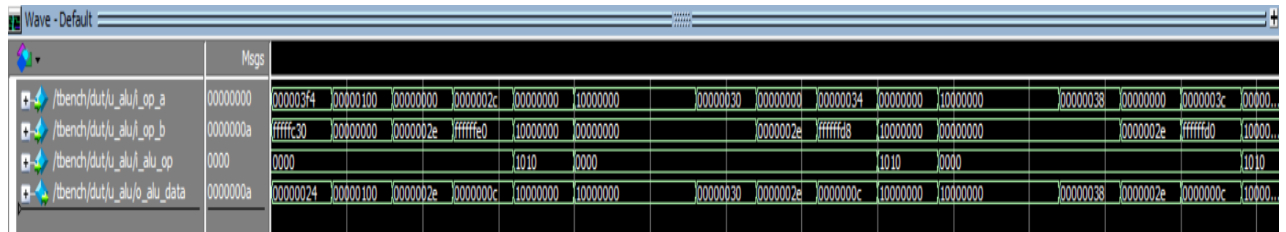


Hình 2.14: Sơ đồ tổng quát khối ALU

o_alu_op	Phép toán	Chức năng
0000	ADD	Phép cộng
0001	SUB	Phép trừ
0010	SLL	Dịch trái logic
0011	SLT	So sánh nhỏ hơn có dấu
0100	SLTU	So sánh nhỏ hơn không dấu
0101	XOR	Phép XOR bit
0110	SRL	Dịch phải logic

0111	SRA	Dịch phải số học
1000	OR	Phép OR bit
1001	AND	Phép AND bit
1010	LUI	Nạp giá trị immediate

Bảng 2.3: Opcode các phép toán khối ALU



Hình 2.15: Waveform khối ALU

Waveform cho thấy khối ALU thực hiện phép toán dựa trên tín hiệu điều khiển `i_alu_op`. Khi `i_alu_op` = 0000 (ALU_ADD), ALU thực hiện phép cộng hai toán hạng `i_op_a` và `i_op_b`, do đó `o_alu_data` = `i_op_a` + `i_op_b`. Ví dụ, với `i_op_a` = 0x34 và `i_op_b` = 0xFFFFFFFF0, kết quả thu được là 0x24. Tương tự, 0x100 + 0 = 0x100 và 0x2C + 0xFFFFFFFFE0 = 0x0C. Khi `i_alu_op` = 1010 (ALU_LUI), ALU đưa giá trị toán hạng immediate tại `i_op_b` ra ngõ ra, thể hiện qua các giá trị 0x10000000 trên waveform.

2.3.7. Branch Comparator (BRC)

Branch Comparator là khối dùng để so sánh giá trị của hai thanh ghi nguồn nhằm phục vụ cho các lệnh rẽ nhánh.

Khối này thực hiện các phép so sánh bằng nhau, khác nhau, nhỏ hơn hoặc lớn hơn dựa trên yêu cầu của từng loại lệnh nhánh. Kết quả so sánh sẽ được gửi đến Control Unit để quyết định xem bộ xử lý có thực hiện nhảy đến địa chỉ mới hay tiếp tục thực hiện tuần tự hay không.



Hình 2.16: Sơ đồ khối BRC

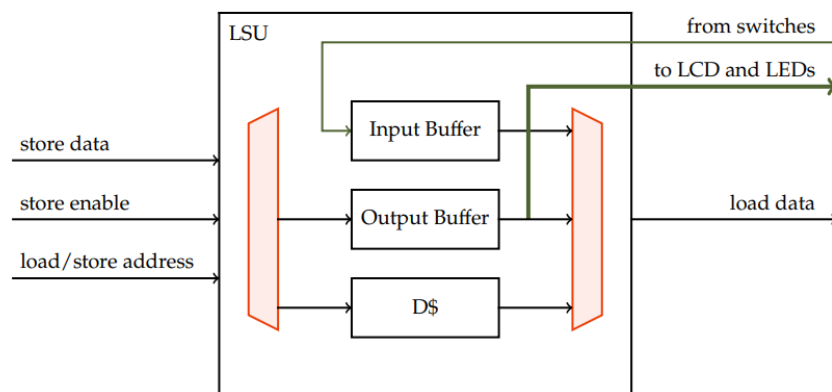
2.3.8. Load Store Unit (LSU)

Load Store Unit (LSU) là khối chịu trách nhiệm xử lý các thao tác truy cập bộ nhớ dữ liệu và thiết bị ngoại vi. Dựa trên địa chỉ truy cập, LSU thực hiện giải mã để xác định vùng dữ liệu tương ứng.

Đối với các lệnh Load, LSU đọc dữ liệu từ bộ nhớ hoặc thiết bị ngoại vi, thực hiện lựa chọn và căn chỉnh dữ liệu theo kích thước truy cập. Với dữ liệu byte hoặc halfword, LSU thực hiện mở rộng dấu hoặc mở rộng zero tùy theo loại lệnh trước khi dữ liệu được đưa về đường Write Back.

Đối với các lệnh Store, LSU thực hiện ghi dữ liệu từ thanh ghi nguồn vào bộ nhớ hoặc các thanh ghi đệm ngoại vi. Khi truy cập bộ nhớ, dữ liệu được căn chỉnh theo địa chỉ và LSU tạo byte mask tương ứng để hỗ trợ các thao tác ghi byte, halfword và word.

Ngoài ra, LSU còn thực hiện giao tiếp với các thiết bị ngoại vi thông qua cơ chế ánh xạ bộ nhớ (memory-mapped I/O), cho phép CPU truy cập các thiết bị ngoại vi bằng các thao tác Load và Store tương tự như khi truy cập bộ nhớ dữ liệu.



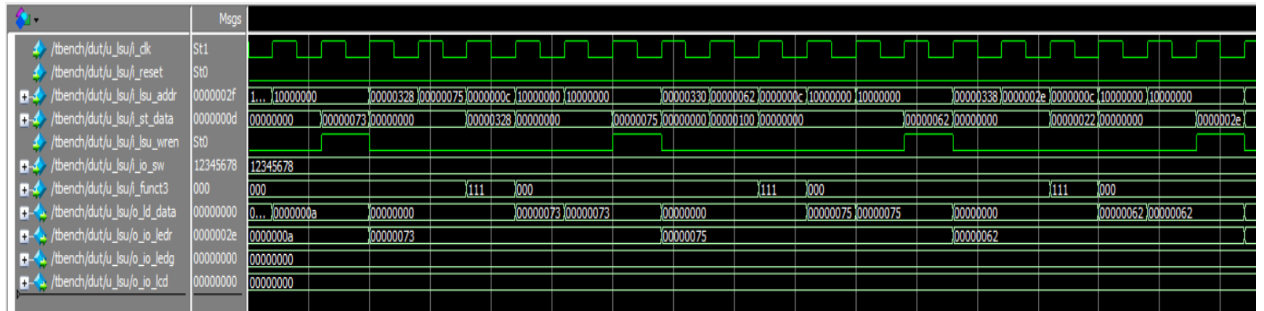
Hình 2.17: Sơ đồ khối LSU

Base address	Top address	Mapping
0x1001_1000	0xFFFF_FFFF	(Reserved)
0x1001_0000	0x1001_0FFF	Switches <i>(required)</i>
0x1000_5000	0x1000_FFFF	(Reserved)
0x1000_4000	0x1000_4FFF	LCD Control Registers
0x1000_3000	0x1000_3FFF	Seven-segment LEDs 7-4
0x1000_2000	0x1000_2FFF	Seven-segment LEDs 3-0
0x1000_1000	0x1000_1FFF	Green LEDs <i>(required)</i>
0x1000_0000	0x1000_0FFF	Red LEDs <i>(required)</i>
0x0000_0800	0x0FFF_FFFF	(Reserved)
0x0000_0000	0x0000_07FF	Memory (2KiB) <i>(required)</i>

Bảng 2.4: LSU Memory Mapping

Tên tín hiệu	Độ rộng (bit)	Input Output	Mô tả
i_clk	1	Input	Tín hiệu xung clock toàn cục, hoạt động tại cạnh lên.
i_reset	1	Input	Tín hiệu reset toàn cục.
i_lsu_addr	32	Input	Địa chỉ dùng để đọc/ghi dữ liệu.
i_st_data	32	Input	Dữ liệu cần ghi.
i_lsu_wren	1	Input	Tín hiệu cho phép ghi (1 khi thực hiện ghi).
o_ld_data	32	Output	Dữ liệu được đọc từ bộ nhớ.
o_io_ledr	32	Output	Dữ liệu ngõ ra điều khiển LED đỏ.
o_io_ledg	32	Output	Dữ liệu ngõ ra điều khiển LED xanh.
o_io_hex0..7	7	Output	Dữ liệu ngõ ra điều khiển các LED 7 đoạn.
o_io_lcd	32	Output	Dữ liệu ngõ ra của thanh ghi LCD.
i_io_sw	32	Input	Dữ liệu ngõ vào từ các công tắc (switch).

Bảng 2.5: Các cổng Input/Output của khối LSU và ngoại vi



Hình 2.18: Waveform khối LSU

Waveform cho thấy khối LSU thực hiện đúng chức năng giải mã địa chỉ và truy cập các vùng nhớ ngoại vi. Khi `i_lsu_addr` thuộc vùng địa chỉ của LED đỏ (`0x10000000`) và `i_lsu_wren = 1`, dữ liệu tại `i_st_data` được ghi vào `output_buffer_ledr` tại cạnh lên của xung clock, làm cho `o_io_ledr` lần lượt nhận các giá trị như `0x00000073`, `0x00000075` và `0x00000062`. Khi `i_lsu_wren = 0`, LSU thực hiện đọc dữ liệu từ vùng được lựa chọn bởi `i_lsu_addr` và đưa kết quả ra `o_ld_data`. Đối với vùng bộ nhớ dữ liệu, `i_func3` xác định kiểu truy cập như LB, LBU, LH, LHU hoặc LW, khi `i_func3 = 111` là giá trị không thuộc các kiểu Load được định nghĩa, LSU sử dụng trường hợp mặc định và trả về toàn bộ `read_data`. Kết quả mô phỏng cho thấy LSU đã thực hiện đúng việc phân biệt vùng nhớ và ngoại vi, điều khiển thao tác đọc/ghi, đồng thời cập nhật ngõ ra LED theo dữ liệu được ghi.

2.3.9. Memory

Memory là khối bộ nhớ dữ liệu được sử dụng để lưu trữ dữ liệu trong quá trình thực thi chương trình. Bộ nhớ dữ liệu cho phép bộ xử lý thực hiện các thao tác đọc và ghi dữ liệu thông qua các lệnh Load và Store thuộc tập lệnh RISC-V.

Trong kiến trúc RISC-V, Memory hoạt động theo mô hình Load/Store Architecture, nghĩa là chỉ các lệnh Load và Store mới được phép truy cập trực tiếp đến bộ nhớ. Các phép toán số học và logic chỉ được thực hiện trên dữ liệu lưu trong các thanh ghi.

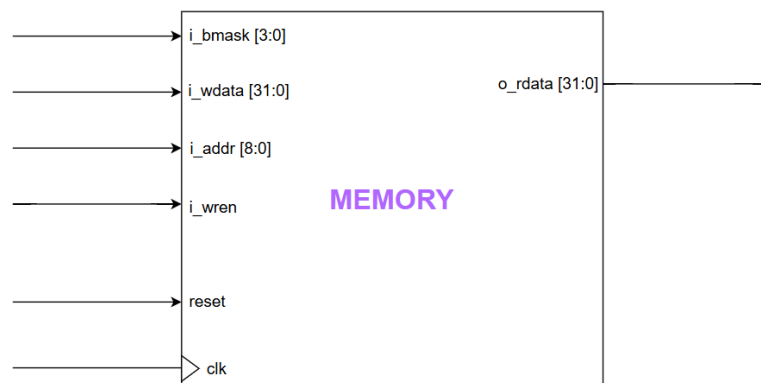
Đối với các lệnh Load, bộ xử lý sẽ đọc dữ liệu từ bộ nhớ và chuyển dữ liệu đó đến Register File để phục vụ cho quá trình xử lý tiếp theo. Ngược lại, đối với các lệnh Store, dữ liệu từ Register File sẽ được ghi xuống bộ nhớ tại địa chỉ xác định.

Tập lệnh RV32I hỗ trợ nhiều kiểu truy cập bộ nhớ với kích thước dữ liệu khác nhau, bao gồm:

- Load Byte.
- Load Halfword.
- Load Word.
- Store Byte.
- Store Halfword.
- Store Word.

Ngoài ra, đối với các lệnh Load có kích thước nhỏ hơn 32 bit, bộ nhớ dữ liệu cần hỗ trợ cơ chế mở rộng dấu (Sign Extension) hoặc mở rộng không dấu (Zero Extension) trước khi dữ liệu được ghi trở lại Register File.

Trong kiến trúc Single-Cycle, Memory nhận địa chỉ truy cập từ ALU và thực hiện thao tác đọc hoặc ghi dữ liệu trong cùng một chu kỳ xung nhịp. Kết quả đọc từ bộ nhớ sẽ được chuyển đến khối Write Back để ghi trở lại thanh ghi đích khi cần thiết.



Hình 2.19: Sơ đồ khối Memory

Chương 3. THIẾT KẾ HỆ THỐNG

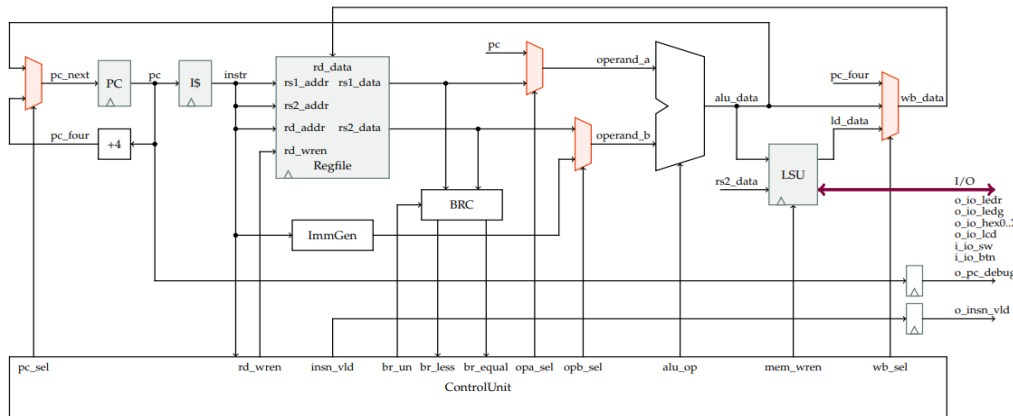
3.1. Tổng quan kiến trúc hệ thống

Trong đề tài này, một bộ xử lý RISC-V đơn nhân dựa trên kiến trúc Single-Cycle được thiết kế và hiện thực nhằm hỗ trợ tập lệnh cơ sở RV32I. Trong kiến trúc Single-Cycle, mỗi lệnh được thực thi hoàn toàn trong một chu kỳ xung nhịp, bao gồm các giai đoạn lấy lệnh, giải mã lệnh, thực thi, truy cập bộ nhớ và ghi kết quả.

Kiến trúc tổng thể của bộ xử lý được xây dựng theo mô hình datapath kết hợp với bộ điều khiển trung tâm (Control Unit). Hệ thống bao gồm các khối chức năng chính như Program Counter (PC), Instruction Memory, Register File (RegFile), Immediate Generator (ImmGen), Arithmetic Logic Unit (ALU), Branch Comparator (BRC), Load Store Unit (LSU), Memory và Control Unit. Các khối này được kết nối với nhau thông qua các đường dữ liệu và tín hiệu điều khiển nhằm thực hiện đầy đủ các chức năng của tập lệnh RV32I.

Ngoài việc thực hiện các phép toán số học và logic cơ bản, bộ xử lý còn hỗ trợ các nhóm lệnh truy cập bộ nhớ, lệnh nhảy và rẽ nhánh, cũng như các lệnh thao tác với hằng số theo đặc tả của tập lệnh RV32I.

Thông qua sự phối hợp giữa các khối chức năng và các tín hiệu điều khiển, bộ xử lý có khả năng thực thi đầy đủ các nhóm lệnh thuộc tập lệnh RV32I trong một chu kỳ xung nhịp, đáp ứng mục tiêu thiết kế của đề tài.

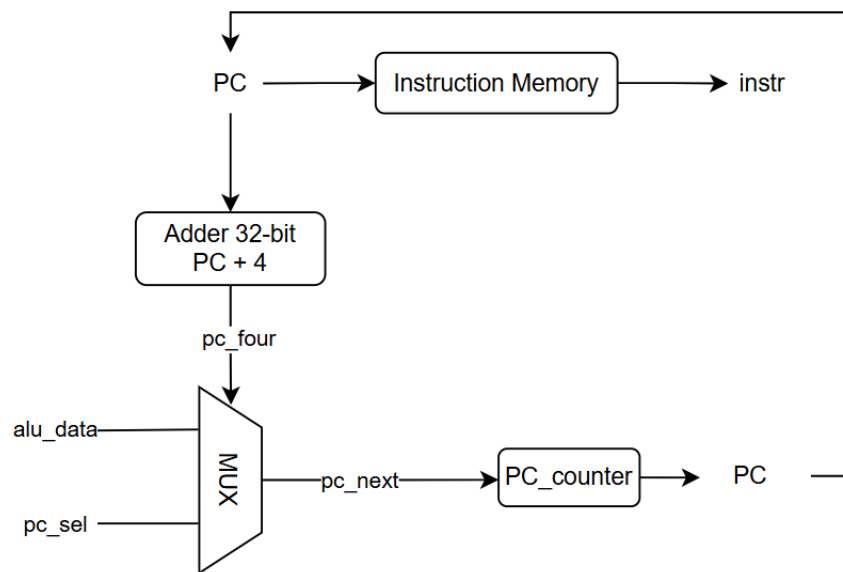


Hình 3.1: Tổng quan kiến trúc hệ thống

3.2. Thiết kế bộ xử lý RISC-V Single-Cycle

3.2.1. Tầng Fetch

Tầng Instruction Fetch là tầng đầu tiên trong quá trình thực thi lệnh của bộ xử lý RISC-V Single-Cycle. Nhiệm vụ của tầng này là xác định địa chỉ của lệnh cần thực thi, truy xuất bộ nhớ lệnh để lấy mã lệnh tương ứng và tính toán địa chỉ của lệnh kế tiếp.



Hình 3.2: Kiến trúc tầng Fetch

Tầng Fetch bao gồm 4 thành phần chính:

- Program Counter (PC)
- Adder 32-bit
- Multiplexer
- Instruction Memory

Quá trình hoạt động của tầng Fetch:

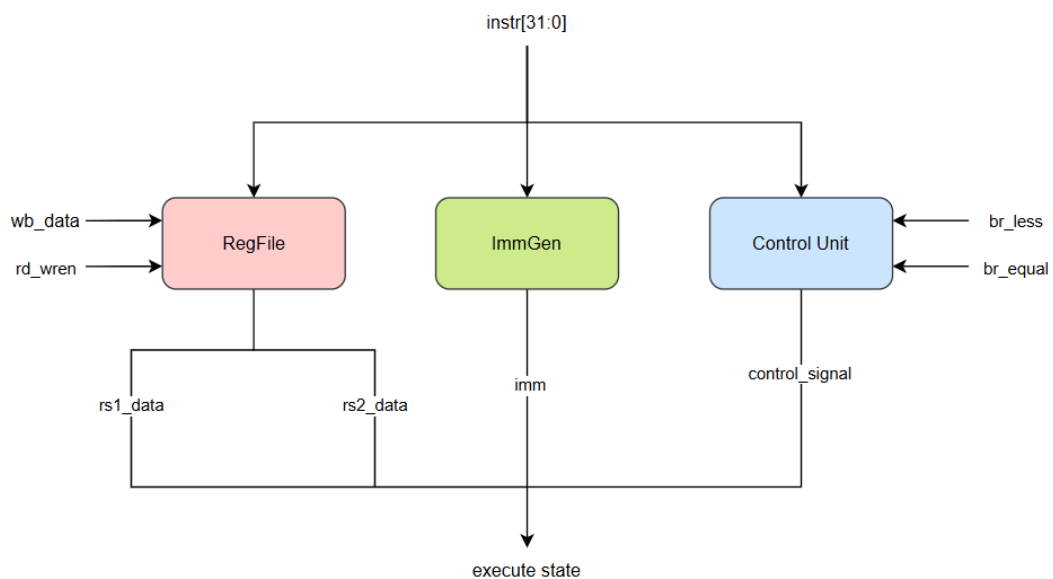
- Program Counter (PC) lưu địa chỉ của lệnh hiện tại
- Giá trị PC được đưa đến Instruction Memory để đọc lệnh tại địa chỉ tương ứng.
- Đồng thời, bộ cộng 32-bit sẽ thực hiện cộng: $PC_{next} = PC + 4$ để tạo địa chỉ của lệnh tiếp theo.

- Mạch MUX lựa chọn địa chỉ kế tiếp sẽ quyết định giá trị nạp vào Program Counter. Nếu chương trình thực hiện tuần tự, địa chỉ mới sẽ là $PC + 4$. Nếu xuất hiện lệnh nhảy hoặc rẽ nhánh, nếu thoả mãn điều kiện, địa chỉ mới sẽ là địa chỉ do ALU tính toán.
- Tại cạnh lên của xung clock tiếp theo, Program Counter sẽ cập nhật giá trị mới và quá trình thực thi của lệnh tiếp theo bắt đầu.

Nhờ cơ chế này, tầng Fetch đảm bảo mỗi chu kì xung nhịp đều cung cấp đúng lệnh cần thực thi cho các tầng tiếp theo.

3.2.2. Tầng Decode

Tầng Instruction Decode là tầng thứ hai trong quá trình thực thi lệnh của bộ xử lý RISC-V Single-Cycle. Sau khi lệnh được lấy từ bộ nhớ lệnh ở tầng Fetch, tầng Decode có nhiệm vụ phân tích cấu trúc của lệnh, đọc dữ liệu từ các thanh ghi nguồn và sinh các tín hiệu điều khiển phục vụ cho các tầng tiếp theo.



Hình 3.3: Kiến trúc tầng Decode

Tầng Decode bao gồm 3 khối chức năng chính:

- Register File
- Immediate Generator (ImmGen)
- Control Unit

Quá trình hoạt động tầng Decode:

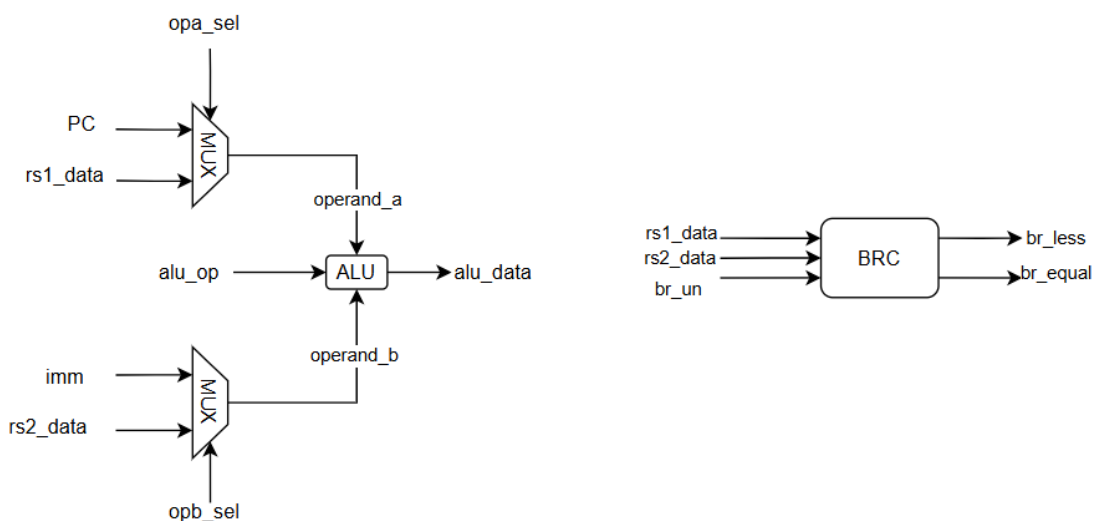
- Lệnh 32 bit được truyền từ Instruction Memory đến tầng Decode.
- Register File sử dụng các trường rs1 và rs2 của lệnh để đọc dữ liệu từ hai thanh ghi nguồn.
- ImmGen phân tích lệnh và tạo ra giá trị immediate 32-bit tương ứng.
- Control Unit giải mã opcode kết hợp với kết quả so sánh từ Branch Comparator để sinh các tín hiệu điều khiển cho toàn bộ datapath.
- Dữ liệu đọc từ Register File, giá trị immediate và các tín hiệu điều khiển được chuyển đến tầng Execute.

Tại tầng Decode, bộ xử lý đã có đầy đủ các dữ liệu và tín hiệu điều khiển cho các phép toán ở tầng tiếp theo.

3.2.3. Tầng Execute

Tầng Execute là tầng thứ ba trong quá trình thực thi lệnh của bộ xử lý RISC-V Single-Cycle. Nhiệm vụ chính của tầng này là thực hiện các phép toán số học, logic, tính toán địa chỉ truy cập bộ nhớ và xác định điều kiện rẽ nhánh của chương trình.

Đầu vào của tầng Execute là dữ liệu từ Register File, giá trị immediate từ Immediate Generator và các tín hiệu điều khiển được sinh bởi Control Unit. Sau khi xử lý, kết quả được chuyển đến tầng Memory Access hoặc tầng Write Back tùy theo loại lệnh.



Hình 3.4: Kiến trúc tầng Execute

Tầng Execute bao gồm các khối chức năng sau:

- Arithmetic Logic Unit (ALU)
- Branch Comparator (BRC)
- Multiplexer

Sau khi tầng Decode hoàn thành việc đọc dữ liệu từ Register File và sinh giá trị immediate, tầng Execute sẽ lựa chọn các toán hạng phù hợp để đưa vào ALU.

Đối với các lệnh số học và logic giữa hai thanh ghi, hai toán hạng của ALU được lấy trực tiếp từ Register File.

Đối với các lệnh sử dụng giá trị immediate hoặc các lệnh truy cập bộ nhớ, toán hạng thứ hai của ALU sẽ được lấy từ khối Immediate Generator.

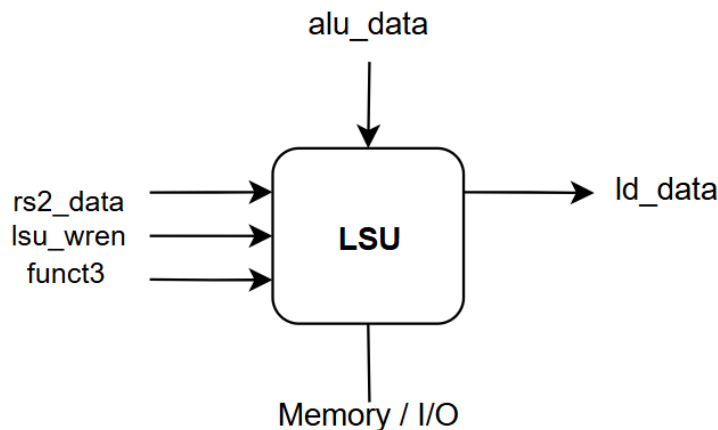
Riêng đối với các lệnh nhảy như JAL hoặc AUIPC, Program Counter được sử dụng làm toán hạng thứ nhất để tính toán địa chỉ đích.

Song song với quá trình thực hiện phép toán tại ALU, khối Branch Comparator tiến hành so sánh dữ liệu của hai thanh ghi nguồn nhằm xác định điều kiện thực hiện các lệnh rẽ nhánh. Kết quả so sánh được trả về Control Unit để quyết định địa chỉ tiếp theo của Program Counter.

3.2.4. Tầng Memory Access

Tầng Memory Access là tầng thứ tư trong quá trình thực thi lệnh của bộ xử lý RISC-V Single-Cycle. Tầng này có nhiệm vụ thực hiện các thao tác đọc và ghi dữ liệu đối với bộ nhớ hoặc các thiết bị ngoại vi thông qua các lệnh Load và Store thuộc tập lệnh RV32I.

Trong bộ xử lý được thiết kế, tầng Memory Access bao gồm khối Load Store Unit (LSU). Khối này tiếp nhận địa chỉ truy cập do ALU tính toán, dữ liệu cần ghi từ Register File và các tín hiệu điều khiển từ Control Unit để thực hiện thao tác truy cập bộ nhớ hoặc ngoại vi.



Hình 3.5: Kiến trúc tầng Memory Access

Sau khi tầng Execute hoàn thành việc tính toán địa chỉ truy cập, địa chỉ này được truyền đến Load Store Unit. Đồng thời, dữ liệu từ thanh ghi nguồn thứ hai rs2 cũng được chuyển đến LSU để phục vụ cho các lệnh ghi dữ liệu xuống bộ nhớ.

Đối với các lệnh Load, LSU sử dụng địa chỉ truy cập để đọc dữ liệu từ Data Memory hoặc từ các thiết bị ngoại vi. Sau khi dữ liệu được xử lý theo đúng kích thước truy cập và kiểu mở rộng dữ liệu, kết quả sẽ được chuyển đến tầng Write Back.

Đối với các lệnh Store, LSU nhận dữ liệu từ Register File và ghi xuống Data Memory hoặc ngoại vi tại địa chỉ tương ứng.

Nhờ việc tập trung toàn bộ chức năng truy cập bộ nhớ vào LSU, datapath của bộ xử lý trở nên đơn giản và thuận tiện cho việc mở rộng trong các phiên bản tiếp theo.

3.2.5. Tầng Write Back

Tầng Write Back là giai đoạn cuối cùng trong quá trình thực thi lệnh của bộ xử lý RISC-V Single-Cycle. Nhiệm vụ của tầng này là lựa chọn dữ liệu phù hợp để ghi trở lại Register File, hoàn tất việc thực thi một lệnh.

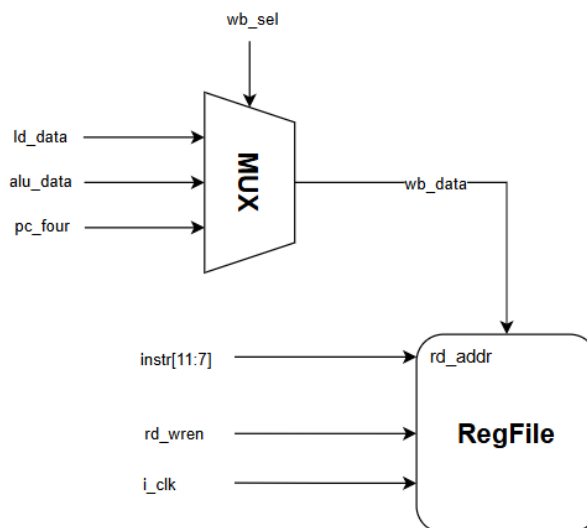
Tùy theo loại lệnh, dữ liệu ghi về thanh ghi đích có thể đến từ nhiều nguồn khác nhau như kết quả tính toán của ALU, dữ liệu đọc từ bộ nhớ hoặc giá trị địa chỉ của lệnh kế tiếp ($PC + 4$).

Trong bộ xử lý được thiết kế, việc lựa chọn dữ liệu ghi về được thực hiện thông qua bộ Multiplexer điều khiển bởi tín hiệu wb_sel do Control Unit sinh ra.

Sau khi hoàn thành tầng Execute hoặc Memory Access, các kết quả xử lý sẽ được đưa đến tầng Write Back. Bộ chọn dữ liệu sẽ lựa chọn một trong ba nguồn dữ liệu để ghi về Register File, bao gồm:

- Kết quả từ ALU (alu_data): sử dụng cho các lệnh số học, tính toán địa chỉ, logic.
- Dữ liệu đọc từ bộ nhớ (ld_data): sử dụng cho các lệnh Load.
- Giá trị $PC + 4$ (pc_four): sử dụng cho các lệnh nhảy nhằm lưu địa chỉ trả về.

Dữ liệu được chọn sẽ dựa trên tín hiệu wb_sel , sau khi dữ liệu được lựa chọn, tín hiệu rd_wren sẽ cho phép ghi dữ liệu vào thanh ghi đích rd trong Register File tại cạnh lên của xung clock.



Hình 3.6: Kiến trúc tầng Write Back

Chương 4. KẾT QUẢ THỰC NGHIỆM

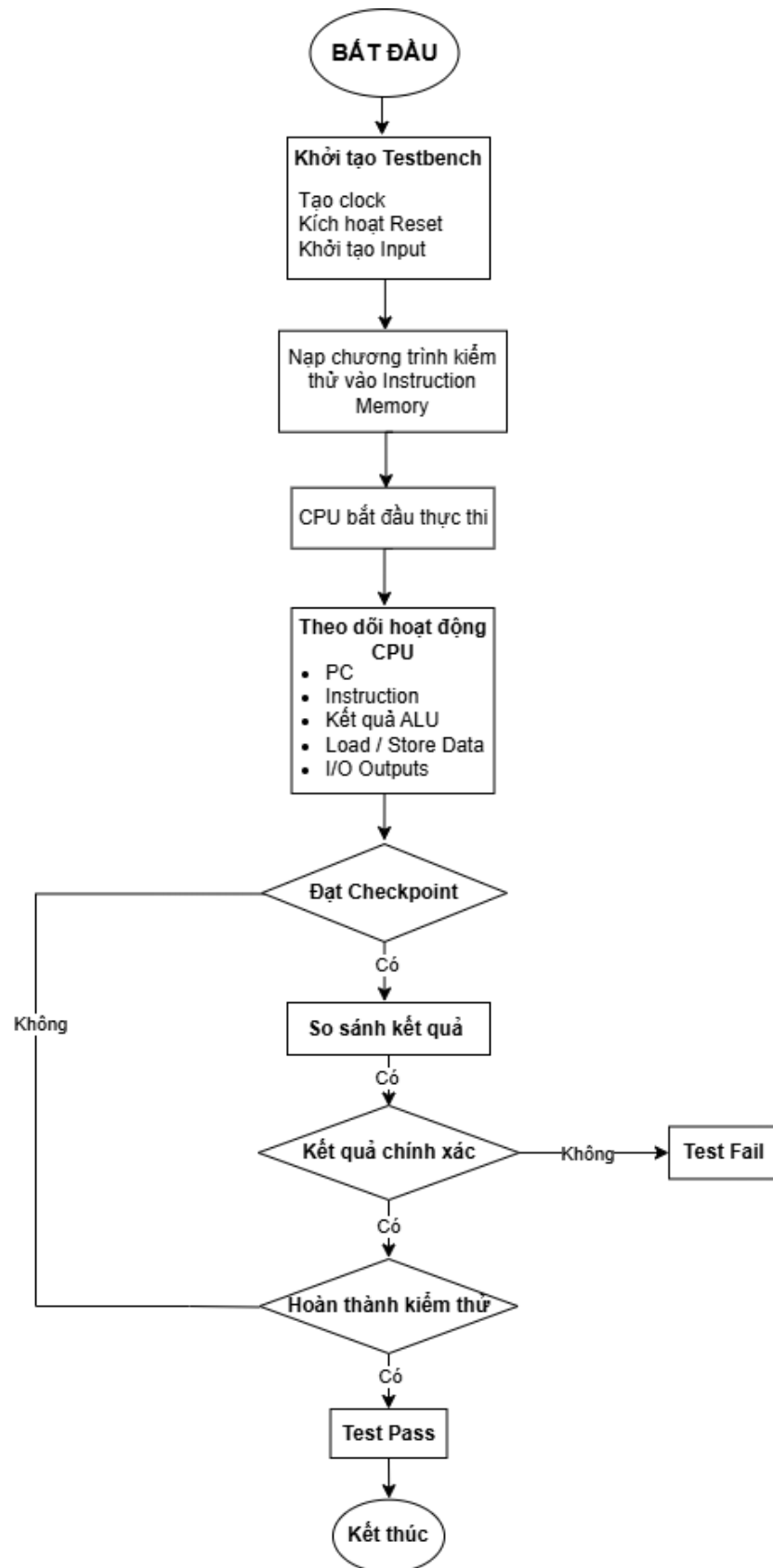
4.1. Môi trường thực nghiệm

Tổng quan thiết kế hệ thống này sẽ được trình bày trong chương này. Thông tin hệ thống sử dụng:

Thành phần	Công cụ sử dụng
Ngôn ngữ phần cứng	Verilog HDL
Trình soạn thảo mã nguồn	Visual Studio Code
Phần mềm mô phỏng	ModelSim Intel FPGA Edition
Hệ điều hành	Windows 11
Kiến trúc bộ xử lý	RISC-V RV32I Single-Cycle

Bảng 4.1: Thông tin hệ thống sử dụng

Trong quá trình mô phỏng, chương trình kiểm thử được biên dịch thành tệp `mem.dump` và nạp vào bộ nhớ lệnh thông qua lệnh `$readmemh`. Sau khi khởi động mô phỏng, CPU sẽ thực thi tuần tự các lệnh trong bộ nhớ và kết quả được quan sát thông qua waveform cũng như các tín hiệu đầu ra của hệ thống.



Hình 4.1: Flowchart thực thi của Testbench

4.2. Kịch bản kiểm thử

Để đánh giá tính đúng đắn của bộ xử lý RISC-V Single-Cycle đã thiết kế, quá trình kiểm thử được thực hiện thông qua ba phiên bản testbench với các mục tiêu khác nhau. Phiên bản thứ nhất sử dụng cấu trúc Driver–Scoreboard để kiểm tra việc thực thi chương trình và tập lệnh RV32I. Phiên bản thứ hai kiểm thử cơ chế Memory-Mapped I/O (MMIO) nhằm đánh giá quá trình trao đổi dữ liệu giữa bộ xử lý và các thiết bị ngoại vi. Phiên bản thứ ba thực hiện chương trình xuất chuỗi ký tự "HELLO" thông qua MMIO, nhằm kiểm chứng khả năng phối hợp của toàn bộ datapath trong quá trình thực thi một chương trình hoàn chỉnh. Tất cả các kịch bản kiểm thử đều được mô phỏng bằng các testbench viết bằng ngôn ngữ Verilog.

4.2.1. Kiểm thử tập lệnh bằng Driver và Scoreboard

Phiên bản testbench thứ nhất được xây dựng theo cấu trúc gồm ba thành phần chính: DUT (Design Under Test), Driver và Scoreboard. Trong đó, DUT là bộ xử lý RISC-V Single-Cycle cần kiểm thử; Driver chịu trách nhiệm tạo dữ liệu đầu vào cho hệ thống; Scoreboard theo dõi quá trình thực thi và kiểm tra kết quả đầu ra.

Testbench tạo tín hiệu clock với chu kỳ xác định và giữ tín hiệu reset ở mức thấp trong giai đoạn đầu mô phỏng. Sau khi reset được giải phóng, bộ xử lý bắt đầu thực thi chương trình đã được nạp vào Instruction Memory thông qua file mem.dump.

Khởi Driver tạo dữ liệu đầu vào mô phỏng cho các thiết bị ngoại vi. Trong kịch bản kiểm thử, giá trị đầu vào của Switch được thiết lập như sau:

- o_sw_data: 32'h12345678;

Dữ liệu này được đưa đến ngõ vào i_io_sw của bộ xử lý và có thể được chương trình kiểm thử truy cập thông qua Load Store Unit.

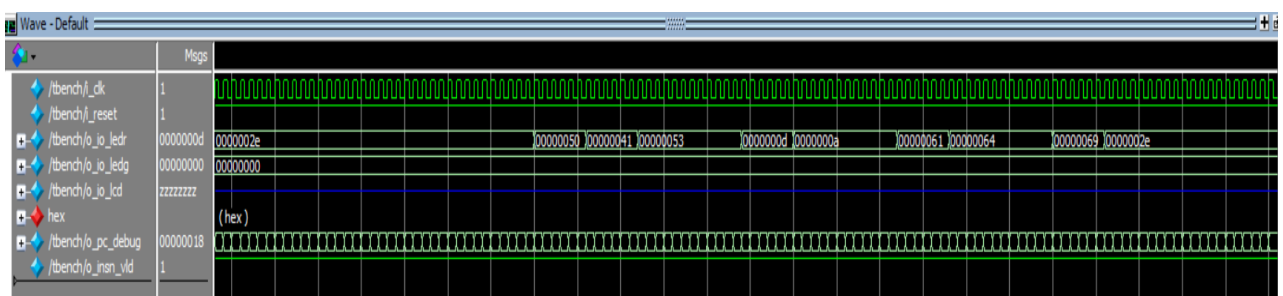
Chương trình ISA test tự thực hiện các phép kiểm tra và tạo thông báo kết quả. Scoreboard theo dõi quá trình thực thi thông qua o_pc_debug; tại địa chỉ xuất dữ liệu, ký tự từ o_io_ledr[7:0] được thu nhận và hiển thị lên cửa sổ Transcript của ModelSim.

Tại địa chỉ 0x18, dữ liệu tại ngõ ra LED đỏ được đọc và hiển thị dưới dạng ký tự. Khi Program Counter đạt địa chỉ 0x1C, Scoreboard xác định chương trình kiểm thử đã hoàn thành và kết thúc quá trình mô phỏng.

Trình tự thực hiện:

1. Khởi tạo tín hiệu clock và reset.
2. Driver cung cấp dữ liệu đầu vào cho bộ xử lý.
3. Chương trình kiểm thử được nạp vào Instruction Memory.
4. Bộ xử lý thực thi tuần tự các lệnh trong chương trình.
5. Scoreboard theo dõi Program Counter và các ngõ ra của hệ thống.
6. Khi đạt đến các địa chỉ kiểm tra, kết quả được hiển thị trên ModelSim.
7. Khi chương trình đạt đến địa chỉ kết thúc, quá trình mô phỏng được dừng.

Kịch bản này được sử dụng để kiểm tra khả năng thực thi chương trình của bộ xử lý và xác nhận rằng các khối chức năng trong datapath có thể phối hợp để thực hiện các lệnh thuộc tập lệnh được hỗ trợ. Hình 4.2 dưới đây thể hiện kết quả mô phỏng của testbench đầu tiên:



Hình 4.2: Waveform kiểm tra testbench1

```

add wave -position insertpoint sim:/tbench/*
VSIM 12> run -all
#
# SINGLE CYCLE - ISA test
#
# add.....PASS
#
# addi.....PASS
#
# sub.....PASS
#
# and.....PASS
#
# andi.....PASS
#
# or.....PASS
#
# ori.....PASS
#
# xor.....PASS
#
# xori.....PASS
#
# slt.....PASS
#
# slti.....PASS
#
# sltu.....PASS
#
# sltiu.....PASS
#
# sll.....PASS
#
# slli.....PASS
#
# srl.....PASS
#
# srli.....PASS
#
# sra.....PASS
#
# srai.....PASS
#
# lw.....PASS
#
#
# lh.....PASS
#
# lhu.....PASS
#
# lb.....PASS
#
# lbu.....PASS
#
# sw.....PASS
#
# sh.....PASS
#
# sb.....PASS
#
# auipc.....PASS
#
# lui.....PASS
#
# beq.....PASS
#
# bne.....PASS
#
# blt.....PASS
#
# bltu.....PASS
#
# bge.....PASS
#
# bgeu.....PASS
#
# jal.....PASS
#
# jalr.....PASS
#
# malgn....ERROR
#
# lsw.....PASS
#
#
# END of ISA test
#
# ** Note: $finish      : D:/NLS_Intern/risc_v/Testbench/scoreboard.v(33)
#    Time: 19422 ps  Iteration: 1  Instance: /tbench/scoreboard
#

```

Hình 4.3: Kết quả mô phỏng

Kết quả mô phỏng cho thấy bộ xử lý thực thi thành công phần lớn các nhóm lệnh được kiểm tra, bao gồm lệnh số học, logic, dịch bit, so sánh, truy cập bộ nhớ, rẽ nhánh và nhảy. Cụ thể, các lệnh ADD, ADDI, SUB, AND, ANDI, OR, ORI, XOR, XORI, SLT, SLTI, SLTU, SLTIU, SLL, SLLI, SRL, SRLI, SRA, SRAI, LW, LH, LHU, LB, LBU, SW, SH, SB, AUIPC, LUI, BEQ, BNE, BLT, BLTU, BGE, BGEU, JAL và JALR đều trả về trạng thái PASS.

Nhóm lệnh	Các lệnh được kiểm tra	Kết quả
Số học	ADD, ADDI, SUB	PASS
Logic	AND, ANDI, OR, ORI, XOR, XORI	PASS
So sánh	SLT, SLTI, SLTU, SLTIU	PASS
Dịch bit	SLL, SLLI, SRL, SRLI, SRA, SRAI	PASS
Load	LW, LH, LHU, LB, LBU	PASS
Store	SW, SH, SB	PASS
Upper Immediate	LUI, AUIPC	PASS
Branch	BEQ, BNE, BLT, BLTU, BGE, BGEU	PASS
Jump	JAL, JALR	PASS
Truy cập I/O	IOSW	PASS
Kiểm tra địa chỉ không căn chỉnh	MALGN	ERROR

Bảng 4.2: Kết quả kiểm thử các lệnh của bộ xử lý RISC-V Single-Cycle

Từ kết quả mô phỏng, có thể kết luận bộ xử lý RISC-V Single-Cycle đã thực thi đúng toàn bộ các lệnh RV32I được triển khai và nằm trong phạm vi hỗ trợ của thiết kế, ngoại trừ chức năng xử lý truy cập không căn chỉnh. Lệnh MALGN không nằm trong tập lệnh mà chương trình có thể thực thi nên kết quả hiển thị ERROR. Kết quả này xác nhận các khối chức năng chính của bộ xử lý đã phối hợp đúng trong quá trình thực thi chương trình.

4.2.2. Kiểm thử tích hợp Memory-Mapped I/O

Phiên bản testbench thứ hai được xây dựng nhằm kiểm tra khả năng hoạt động của toàn bộ bộ xử lý thông qua một luồng dữ liệu hoàn chỉnh. Khác với phiên bản thứ nhất, testbench này không sử dụng module Driver và Scoreboard riêng biệt. Việc tạo dữ liệu đầu vào, theo dõi tín hiệu và so sánh kết quả được thực hiện trực tiếp bên trong testbench.

Mục tiêu của kịch bản kiểm thử là kiểm tra luồng dữ liệu:

Switch → LSU → Register File → LSU → LEDR

Kiểm thử đánh giá sự phối hợp của các khối Instruction Memory, Control Unit, Register File, ALU, LSU và cơ chế Write Back.

Chương trình kiểm thử được lưu trong file mem_mmio.dump và nạp vào Instruction Memory. Chương trình gồm năm lệnh:

Địa chỉ PC	Lệnh	Chức năng
0x00	LUI x1, 0x10010	Tạo địa chỉ vùng Switch
0x04	LW x2, 0(x1)	Đọc dữ liệu vùng Switch vào thanh ghi x2
0x08	LUI x3, 0x10000	Tạo địa chỉ vùng LEDR
0x0C	SW x2, 0(x3)	Ghi dữ liệu từ x2 ra LEDR
0x10	JAL x0, 0	Giữ chương trình tại vị trí kết thúc

Bảng 4.3: Chương trình kiểm thử truy cập I/O thông qua cơ chế Memory-Mapped I/O

Quá trình kiểm thử:

Bước 1 – Fetch và Decode lệnh tạo địa chỉ Switch: Testbench kiểm tra Program Counter có giá trị 0x00000000, mã lệnh lấy từ Instruction Memory có đúng là 0x100100B7 và tín hiệu o_insn_vld xác nhận đây là một lệnh hợp lệ. Nếu một trong các giá trị không đúng, quá trình mô phỏng được kết thúc với trạng thái thất bại.

Bước 2 – Đọc dữ liệu từ vùng MMIO: Bộ xử lý thực thi lệnh LW x2, 0(x1).

Testbench kiểm tra dữ liệu thanh ghi x1, địa chỉ do ALU tính toán và dữ liệu trả về từ LSU. Kết quả mong đợi là:

$$x1 = 0x10010000$$

$$ALU\ address = 0x10010000$$

$$LSU\ output = 0x12345678$$

Bước 3 – Fetch và Decode lệnh tạo địa chỉ LEDR: Bộ xử lý thực thi lệnh LUI x3, 0x10000. Testbench kiểm tra mã lệnh và tín hiệu xác nhận lệnh hợp lệ. Lệnh này tạo địa chỉ 0x10000000, tương ứng với vùng địa chỉ của LEDR trong hệ thống.

Bước 4 – Ghi dữ liệu ra vùng MMIO: Bộ xử lý thực thi lệnh SW x2, 0(x3).

Testbench kiểm tra địa chỉ cơ sở trong thanh ghi x3, dữ liệu cần ghi trong thanh ghi x2 và địa chỉ do ALU tính toán. Các giá trị mong đợi là:

$$x3 = 0x10000000$$

$$x2 = 0x12345678$$

$$ALU\ address = 0x10000000$$

Bước 5 – Kiểm tra kết quả cuối cùng: Sau khi hoàn thành lệnh Store, testbench so sánh giá trị đầu ra LEDR với giá trị đầu vào Switch:

$$Switch\ input = 0x12345678$$

$$LEDR\ output = 0x12345678$$

Nếu hai giá trị bằng nhau, chương trình sẽ hiển thị:

MMIO TEST PASSED

Hình ảnh dưới đây là kết quả khi chạy testbench:

```

add wave -position insertpoint sim:/tb_singlecycle_mmio/*
vSIM6> run -all
#
# =====
#           RISC-V SINGLE-CYCLE MMIO INTEGRATION TEST
# =====
# Switch input = 0x12345678
# =====
#
# [CPU TRACE] PC=00000000 | INSTR=100100b7 | VALID=1
#
# [STEP 1] FETCH / DECODE
# PC          : 0x00000000
# Instruction  : 0x100100b7
# Assembly    : LUI x1, 0x10010
# Expected    : x1 = 0x10010000
# Purpose     : Load SWITCH address
# [PASS] Instruction fetched and decoded correctly
#
# [CPU TRACE] PC=00000004 | INSTR=0000a103 | VALID=1
#
# [STEP 2] LOAD FROM MMIO
# Instruction  : LW x2, 0(x1)
# rs1_data    : 0x10010000
# ALU address  : 0x10010000
# Switch data  : 0x12345678
# LSU output   : 0x12345678
# [PASS] SWITCH read correctly
#
# [CPU TRACE] PC=00000008 | INSTR=100001b7 | VALID=1
#
# [STEP 3] FETCH / DECODE
# Instruction  : LUI x3, 0x10000
# Expected    : x3 = 0x10000000
# Purpose     : Load LEDR address
# [PASS] Instruction fetched and decoded correctly
#
# [CPU TRACE] PC=0000000c | INSTR=0021a023 | VALID=1
#
# [STEP 4] STORE TO MMIO
# Instruction  : SW x2, 0(x3)
# rs1_data    : 0x10000000
# rs2_data    : 0x12345678
# ALU address  : 0x10000000
# Write data   : 0x12345678
# [PASS] STORE address and data are correct
#
# [CPU TRACE] PC=00000010 | INSTR=0000006f | VALID=1
#
# [STEP 5] FINAL RESULT CHECK
# Switch input : 0x12345678
# LEDR output  : 0x12345678
#
# =====
#           MMIO TEST PASSED
# =====

```

Hình 4.4: Kết quả mô phỏng cơ chế Memory-Mapped I/O

Dựa theo kết quả hiển thị, bộ xử lý đã thực thi đúng với chương trình được nạp vào Instruction Memory. Dữ liệu được đọc từ Switch thông qua LSU, ghi vào RegFile, sau đó được đọc lại từ RegFile và ghi đến LEDR thông qua LSU.

Thông qua quá trình này, có thể xác nhận các khối Instruction Memory, Program Counter, Control Unit, Register File, ALU, LSU và cơ chế Write Back đã phối hợp đúng theo datapath được thiết kế. Kết quả ‘MMIO TEST PASSED’ cho thấy bộ xử lý RISC-V Single-Cycle đã thực hiện thành công quá trình lấy lệnh, giải mã, thực thi, truy cập ngoại vi và ghi kết quả.

4.2.3. Kiểm thử in chuỗi HELLO

Kịch bản kiểm thử thứ ba được xây dựng nhằm đánh giá khả năng thực thi một chương trình hoàn chỉnh của bộ xử lý RISC-V Single-Cycle. Chương trình kiểm thử được nạp vào Instruction Memory, có nhiệm vụ lần lượt tạo các ký tự H, E, L, L, O dưới dạng mã ASCII và ghi chúng đến vùng địa chỉ LEDR thông qua cơ chế Memory-Mapped I/O.

Trong quá trình mô phỏng, testbench theo dõi các tín hiệu nội bộ quan trọng của bộ xử lý như PC, mã lệnh, dữ liệu thanh ghi, toán hạng ALU, kết quả ALU, tín hiệu ghi LSU và dữ liệu Write Back. Tại mỗi lệnh SW, testbench thu nhận dữ liệu ghi ra MMIO, chuyển đổi sang ký tự và so sánh với chuỗi mong đợi.

Địa chỉ PC	Mã máy	Lệnh Assembly	Ý nghĩa
0x00	100000B7	LUI x1, 0x10000	x1 = 0x10000000
0x04	04800113	ADDI x2, x0, 0x48	x2 = 0x48 = 'H'
0x08	0020A023	SW x2, 0(x1)	Ghi H vào địa chỉ 0x10000000
0x0C	04500113	ADDI x2, x0, 0x45	x2 = 0x45 = 'E'
0x10	0020A023	SW x2, 0(x1)	Ghi E vào địa chỉ 0x10000000
0x14	04C00113	ADDI x2, x0, 0x4C	x2 = 0x4C = 'L'
0x18	0020A023	SW x2, 0(x1)	Ghi L vào địa chỉ 0x10000000
0x1C	04C00113	ADDI x2, x0, 0x4C	x2 = 0x4C = 'L'
0x20	0020A023	SW x2, 0(x1)	Ghi L vào địa chỉ 0x10000000
0x24	04F00113	ADDI x2, x0, 0x4F	x2 = 0x4F = 'O'
0x28	0020A023	SW x2, 0(x1)	Ghi O vào địa chỉ 0x10000000
0x2C	0000006F	JAL x0, 0	Lặp vô hạn

Bảng 4.4: Chương trình kiểm thử hiển thị chuỗi "HELLO"

```

# =====
# TIME          = 26000
# PC            = 00000008
# INSTRUCTION   = 0020a023
# RS1_DATA      = 10000000
# RS2_DATA      = 00000048
# IMMEDIATE     = 00000000
# OPERAND_A     = 10000000
# OPERAND_B     = 00000000
# ALU_OP        = 0000
# ALU_DATA      = 10000000
# LSU_ADDR      = 10000000
# STORE_DATA    = 00000048
# LSU_WREN      = 1
# LD_DATA       = 00000000
# WB_SEL        = 01
# WB_DATA       = 10000000
# RD_WREN       = 0
# INSN_VALID    = 1
# =====
#
# PC            = 00000008
# INSTRUCTION   = 0020a023
# LSU_ADDR      = 10000000
# STORE_DATA    = 00000048
# ASCII         = 48
# CHARACTER     = H
# RESULT       = PASS

```

Hình 4.5: Kết quả hiển thị ký tự “H”

```

# =====
# TIME          = 34000
# PC            = 00000010
# INSTRUCTION   = 0020a023
# RS1_DATA      = 10000000
# RS2_DATA      = 00000045
# IMMEDIATE     = 00000000
# OPERAND_A     = 10000000
# OPERAND_B     = 00000000
# ALU_OP        = 0000
# ALU_DATA      = 10000000
# LSU_ADDR      = 10000000
# STORE_DATA    = 00000045
# LSU_WREN      = 1
# LD_DATA       = 00000048
# WB_SEL        = 01
# WB_DATA       = 10000000
# RD_WREN       = 0
# INSN_VALID    = 1
# =====
# PC            = 00000010
# INSTRUCTION   = 0020a023
# LSU_ADDR      = 10000000
# STORE_DATA    = 00000045
# ASCII         = 45
# CHARACTER     = E
# RESULT        = PASS

```

Hình 4.6: Kết quả hiển thị ký tự “E”

```

# =====
# TIME          = 42000
# PC            = 00000018
# INSTRUCTION   = 0020a023
# RS1_DATA      = 10000000
# RS2_DATA      = 0000004c
# IMMEDIATE     = 00000000
# OPERAND_A     = 10000000
# OPERAND_B     = 00000000
# ALU_OP        = 0000
# ALU_DATA      = 10000000
# LSU_ADDR      = 10000000
# STORE_DATA    = 0000004c
# LSU_WREN      = 1
# LD_DATA       = 00000045
# WB_SEL        = 01
# WB_DATA       = 10000000
# RD_WREN       = 0
# INSN_VALID    = 1
# =====
# PC            = 00000018
# INSTRUCTION   = 0020a023
# LSU_ADDR      = 10000000
# STORE_DATA    = 0000004c
# ASCII         = 4c
# CHARACTER     = L
# RESULT        = PASS

```

Hình 4.7: Kết quả hiển thị ký tự “L”

```

# =====
# TIME          = 50000
# PC            = 00000020
# INSTRUCTION   = 0020a023
# RS1_DATA      = 10000000
# RS2_DATA      = 0000004c
# IMMEDIATE     = 00000000
# OPERAND_A     = 10000000
# OPERAND_B     = 00000000
# ALU_OP        = 0000
# ALU_DATA      = 10000000
# LSU_ADDR      = 10000000
# STORE_DATA    = 0000004c
# LSU_WREN      = 1
# LD_DATA       = 0000004c
# WB_SEL        = 01
# WB_DATA       = 10000000
# RD_WREN       = 0
# INSN_VALID    = 1
# =====
# PC            = 00000020
# INSTRUCTION   = 0020a023
# LSU_ADDR      = 10000000
# STORE_DATA    = 0000004c
# ASCII         = 4c
# CHARACTER     = L
# RESULT        = PASS

```

Hình 4.8: Kết quả hiển thị ký tự “L”

```

# =====
# TIME          = 58000
# PC            = 00000028
# INSTRUCTION   = 0020a023
# RS1_DATA      = 10000000
# RS2_DATA      = 0000004f
# IMMEDIATE     = 00000000
# OPERAND_A     = 10000000
# OPERAND_B     = 00000000
# ALU_OP        = 0000
# ALU_DATA      = 10000000
# LSU_ADDR      = 10000000
# STORE_DATA    = 0000004f
# LSU_WREN      = 1
# LD_DATA       = 0000004c
# WB_SEL        = 01
# WB_DATA       = 10000000
# RD_WREN       = 0
# INSN_VALID    = 1
# =====
# PC            = 00000028
# INSTRUCTION   = 0020a023
# LSU_ADDR      = 10000000
# STORE_DATA    = 0000004f
# ASCII         = 4f
# CHARACTER     = 0
# RESULT        = PASS

```

Hình 4.9: Kết quả hiển thị ký tự “O”

```

# =====
# TIME          = 62000
# PC            = 0000002c
# INSTRUCTION   = 0000006f
# RS1_DATA      = 00000000
# RS2_DATA      = 00000000
# IMMEDIATE     = 00000000
# OPERAND_A     = 0000002c
# OPERAND_B     = 00000000
# ALU_OP        = 0000
# ALU_DATA      = 0000002c
# LSU_ADDR      = 0000002c
# STORE_DATA    = 00000000
# LSU_WREN      = 0
# LD_DATA       = 00000000
# WB_SEL        = 10
# WB_DATA       = 00000030
# RD_WREN       = 1
# INSN_VALID    = 1
# =====
# WRITEBACK UPDATED
# TIME          = 62000
# PC            = 0000002c
# WB_DATA       = 00000030
# RD_ADDR       = 0
#
# HELLO PROGRAM PASSED
# EXPECTED STRING = HELLO
# ACTUAL STRING   = HELLO** Note: $finish      : D:/NLS_Intern/risc_v/Testbench/tbench.sv(949)
# Time: 62 ns Iteration: 1 Instance: /tb_singlecycle_hello

```

Hình 4.10: Kết quả hoàn thành mô phỏng

Đối với ký tự đầu tiên H, tại địa chỉ PC = 0x00000008, bộ xử lý thực thi lệnh SW x2, 0(x1) có mã máy 0x0020A023. Register File cung cấp rs1_data = 0x10000000 làm địa chỉ cơ sở và rs2_data = 0x00000048, tương ứng với mã ASCII của ký tự H. ALU tính được địa chỉ truy cập 0x10000000, sau đó LSU thực hiện thao tác ghi khi tín hiệu LSU_WREN = 1. Testbench kiểm tra dữ liệu và trả về kết quả PASS.

Kết quả cho thấy cả năm ký tự đều được xử lý đúng thứ tự và nhận trạng thái PASS. Qua đó, có thể xác nhận bộ xử lý đã thực hiện chính xác chuỗi hoạt động gồm lấy lệnh, giải mã lệnh, đọc Register File, thực hiện phép toán tại ALU và ghi dữ liệu thông qua LSU. Kết quả kiểm thử cho thấy các khối chức năng chính của kiến trúc RISC-V Single-Cycle đã phối hợp đúng trong quá trình thực thi một chương trình hoàn chỉnh.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Sau khi hoàn thành Đồ án 1, nhóm đã nghiên cứu và hiện thực thành công bộ xử lý RISC-V Single-Cycle sử dụng tập lệnh cơ sở RV32I. Bộ xử lý được xây dựng từ các khối chức năng chính gồm Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, Branch Comparator, ALU và Load Store Unit.

Kết quả của Đồ án 1 là cơ sở để tiếp tục phát triển kiến trúc bộ xử lý trong các giai đoạn tiếp theo, hướng đến mục tiêu xây dựng hệ thống RISC-V đa nhân và nghiên cứu cơ chế đồng nhất bộ nhớ đệm theo định hướng của đề tài.

5.2. Hướng phát triển trong tương lai

Trong tương lai, nhóm sẽ tiếp tục phát triển các nội dung sau:

- Hoàn thiện và mở rộng khả năng kiểm thử bộ xử lý RISC-V Single-Cycle.
- Phát triển hệ thống từ kiến trúc đơn nhân lên kiến trúc RISC-V đa nhân.
- Bổ sung bộ nhớ đệm riêng cho từng lõi xử lý nhằm nâng cao hiệu năng truy cập dữ liệu.
- Nghiên cứu và hiện thực cơ chế đồng nhất bộ nhớ đệm giữa các lõi xử lý.
- Xây dựng bộ điều khiển giao tiếp và trao đổi thông tin giữa các bộ nhớ đệm trong hệ thống đa nhân.
- Xây dựng các chương trình kiểm thử nhằm đánh giá tính đúng đắn của cơ chế đồng nhất dữ liệu khi nhiều lõi cùng truy cập vùng nhớ dùng chung.
- Đánh giá hệ thống đa nhân về tài nguyên phần cứng và khả năng hoạt động sau khi hoàn thiện thiết kế.

Sau khi hoàn thành kiến trúc RISC-V đa nhân với cơ chế đồng nhất bộ nhớ đệm, nhóm sẽ tiến hành kiểm thử toàn bộ hệ thống với các chương trình có khả năng khai thác nhiều lõi xử lý, từ đó đánh giá tính đúng đắn và hiệu quả của kiến trúc được đề xuất.

TÀI LIỆU THAM KHẢO

- [1] RISC-V International, “The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 20250508”.
- [2] Ektha Reddy, “RISC-V-Single-Cycle-Processor”,
<https://github.com/EkthaReddy/RISC-V-Single-Cycle-Processor>. August, 2024.
- [3] B. Maneesha, G. Sriram, G. Sai, M. Siddhu, and E. Jaya, “Design of RISC-V Processor Using Verilog”, International Journal of Research Publication and Reviews, vol. 5, no. 4, pp. 2097 2116, Apr. 2024.
- [4] J. -Y. Lai, C. -A. Chen, S. -L. Chen and C. -Y. Su, “Implement 32-bit RISC-V Architecture Processor using Verilog HDL”, 2021 International Symposium on Intelligent Signal Processing and Communication Systems (ISPACS), Hualien City, Taiwan, 2021.
- [5] Keerthi M, Lakshmi Girish, Priyadarshini N M, Dr. Kiran Bailey, “Implementation of Single Cycle Risc-V Processor using Verilog”, Department of Electronics and Communication, B.M.S College of Engineering, Bengaluru, India, 2025.